

Beyond Method Signatures: Resolving Privacy APIs and Asynchronous Flows in ArkTS

Anonymous Author(s)

Abstract

Existing taint analysis frameworks for OpenHarmony/ArkTS assume that privacy-sensitive API sources are already identified. In practice, method-signature-based source configuration misses 31.3% of real API usages: manager-style indirect calls and privacy-constant field reads are invisible to signature matching. HapFlow, the state-of-the-art ArkTS taint analysis, solves *propagation* but not *source identity resolution*—which API calls are privacy-sensitive and how their identities survive package migration, receiver factories, and IR lowering.

We present ARKPRISM, a static analysis framework addressing this bottleneck. ARKPRISM contributes: (1) *package- and receiver-aware API identity resolution*, resolving package-family equivalence, import aliases, IR-lowered aliases, and receiver-origin provenance—two access patterns that are structurally invisible to method-signature matching; (2) *asynchronous source binding*, formalizing how callbacks, field-stored callbacks, builder-option callbacks, and Promise/async-await chains connect sources to downstream consumers, improving taint flow recall from 51.5% to 97.8%; and (3) *provenance-annotated evidence subgraphs*, where each edge records its derivation, enabling auditors to distinguish proven paths from heuristic completions.

On the ArkPrismBench stratified benchmark (164 projects sampled from a 1,015-project corpus), ARKPRISM achieves 100% precision and 100% recall for API recognition; a micro-benchmark of 77 test cases across 15 pattern categories confirms 93.2% positive-case detection. Ablation shows that callback analysis is the dominant component, improving taint flow recall from 51.5% to 97.8% ($p < 0.001$). The full pipeline analyzes 1,015 projects in ≈ 4.2 h wall time with 99.8% completion.

CCS Concepts

• Security and privacy → Software security engineering; • Software and its engineering → Software defect analysis.

Keywords

OpenHarmony, ArkTS, static analysis, privacy APIs, taint analysis, call-chain analysis

ACM Reference Format:

Anonymous Author(s). 2026. Beyond Method Signatures: Resolving Privacy APIs and Asynchronous Flows in ArkTS. In *Proceedings of ACM Conference (Conference'26)*. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

1 Introduction

OpenHarmony has moved from an emerging operating system project to a fast-growing application ecosystem for phones, tablets, wearables, vehicles, and distributed devices. Its native applications are commonly written in ArkTS, a TypeScript-derived language

with stricter typing, declarative ArkUI components, lifecycle callbacks, and a large family of system APIs exposed through both historical @ohos.* packages and newer @kit.* packages [28, 32, 39]. These ecosystem properties create a timely software engineering problem: developers, platform auditors, and marketplace operators need scalable tools to recognize privacy-sensitive API usage and map where privacy-related information may flow.

This problem is especially urgent because privacy-sensitive behavior in OpenHarmony apps is not limited to obviously dangerous calls. An app may read hardware identifiers, query network state, access clipboard contents, subscribe to sensors, obtain location, inspect camera metadata, or create media assets through APIs that are legitimate in isolation. Whether the behavior is risky depends on where the data is obtained, how it is triggered, whether the result is displayed, logged, stored, returned, or sent over the network, and how multiple sources are combined inside a component. Thus, a useful analysis must preserve context. A flat list of sensitive calls is too shallow for auditing, while a taint-flow-only report can hide the package, permission, lifecycle, and source-code evidence that an auditor needs to validate the result.

Privacy analysis for OpenHarmony apps is still young. ArkAnalyzer provides the first reusable static analysis infrastructure for ArkTS [15]; HapFlow builds an IFDS-based taint analysis framework on top of it [16]; other efforts explore testing, repair, and pointer analysis [14, 17, 32–34]. However, these efforts assume that privacy-sensitive API sources are already correctly identified. HapFlow solves *propagation*—whether a configured source reaches a sink—but its source configuration (Json2ArkMethodSignature) matches by namespace and method name against SDK declarations, and therefore cannot recognize APIs invoked through manager/helper receivers or accessed as privacy-constant fields. This is an architectural blind spot: method-signature matching is fundamentally unable to express these patterns. An auditor needs to know which privacy API was used, what package justified the match, what context reaches the API, and whether the evidence is proven or heuristically inferred.

Applying these foundations to a large and heterogeneous corpus exposes three recurring challenges in *source identity resolution*. **C1: API identity is unstable under package migration.** A package migration from @ohos.deviceInfo to @kit.BasicServicesKit can make a strict rule miss a real device-identity access, while method-only matching is too noisy. **C2: Receiver-origin and property identities are invisible to method-signature matching.** ArkAnalyzer's IR lowering can split request.agent.create into temporary variables, and manager-style indirect calls (cameraManager.createCaptureSession()) have no corresponding SDK namespace entry for the receiver. Privacy-constant field reads (deviceInfo.brand) are not method calls at all. **C3: Asynchronous source bindings are implicit.** ArkUI callbacks may be stored in fields or anonymous builder

options before they are bound to user-interface events; Promise .then() chains and async-await bridges create data-flow edges that ordinary call-graph construction does not express. These challenges motivate a system whose contribution is not replacing ArkAnalyzer or HapFlow, but solving the source identity resolution bottleneck that precedes taint propagation.

To see why the source identity bottleneck is critical, consider a concrete failure pattern. An app obtains a camera manager via `camera.getCameraManager(ctx)`, then calls `cameraManager.createCaptureSession()`. HapFlow can recognize `getCameraManager` because the namespace and method appear in SDK declarations, but `createCaptureSession` is called on the *receiver object*, which has no SDK namespace entry—so no source fact is created and the IFDS solver cannot propagate taint from it. ArkPrism resolves this through receiver-origin tracing: it traces the receiver back to `getCameraManager`, identifies the enclosing `@kit.CameraKit` namespace, and creates a source fact under the resolved namespace. The bottleneck is *before* taint propagation: without correct source identity, no solver improvement can recover the missing flows.

This paper targets that gap through the lens of *provenance-annotated evidence subgraphs*. An information-flow subgraph is an explainable slice of an application that connects four kinds of evidence: (1) a privacy-sensitive API usage, (2) the calling context that reaches the usage, (3) downstream exposure points such as logs, UI rendering, storage, network, or data returns, and (4) semantic metadata such as permission requirements, profiling categories, and source snippets. Crucially, each edge records its *provenance*—whether it was derived from call-graph edges, callback resolution, lifecycle model, IFDS propagation, or heuristic association—so that auditors can distinguish semantically proven paths from heuristic completions. The goal is not to replace taint analysis; instead, the goal is to make privacy-related behavior auditable at scale by combining API identity resolution, call-chain recovery, sink characterization, and taint propagation into a unified, provenance-tracked graph representation.

The subgraph view gives us a disciplined way to avoid two common overclaims. First, it avoids equating the existence of a sensitive API with a privacy leak: the API node is only one part of the evidence. Second, it avoids equating every taint path with a confirmed violation: sink type, trigger context, source category, and project role still matter. By preserving these distinctions, ArkPRISM supports both automated measurement and manual triage.

We present ArkPRISM, a static analysis framework for ArkTS privacy-sensitive API identity resolution and provenance-annotated evidence subgraph mapping. ArkPRISM is built on top of ArkAnalyzer and incorporates HapFlow-style IFDS taint analysis, with three core contributions: (1) package- and receiver-aware API identity resolution that computes a semantic identity for each candidate usage, handling direct calls, assigned returns, manager-style indirect calls, and privacy-constant field reads while normalizing package-family equivalence; (2) asynchronous source binding that resolves callback parameters, field-stored callbacks, builder-option callbacks, Promise chains, and async/await bridges; and (3) provenance-annotated evidence subgraphs where each edge records its derivation. Supporting mechanisms—lifecycle modeling

and context-sensitive CG—are available as configuration options but are not the primary contributions.

We evaluate ArkPRISM on a large-scale corpus of 1,015 OpenHarmony/ArkTS projects (31,433 source files, 238,271 methods), detecting 1,757 privacy-sensitive API usages in 236 projects, 1,352 downstream sinks, and 1,792 taint flows. On the ArkPrismBench stratified benchmark (164 projects, including 40 zero-output projects) and ArkPrismBench-Micro (77 test cases, 15 categories), ArkPRISM achieves 100%/100% precision/recall and 93.2% positive-case detection. Ablation shows: callback analysis improves taint flow recall from 51.5% to 97.8% ($p < 0.001$); lifecycle modeling enriches structural understanding (collaboration behavior +44.2% in quantity, precision not systematically audited) but does not increase taint flow counts; k -limited context-sensitive CG provides marginal precision improvement at 5.02× overhead. The full pipeline analyzes 1,015 projects in ≈ 4.2 h wall time (2 workers) with 99.8% completion. The evaluation is intentionally conservative: we do not treat every reported sink as a violation, and note that the corpus contains many capability demos, so ArkPRISM provides an evidence layer for scalable audit while final security-impact claims require human review.

This paper makes the following contributions:

- **Package- and receiver-aware API identity resolution.** We formulate recognizing privacy-sensitive API usages as computing $Id(v) = \langle Pkg, Ns, Recv, Mem, Acc \rangle$ for each candidate usage v , unifying direct invoke, assigned invoke, indirect invoke (manager/helper receivers), and privacy-constant field access. Receiver origin and privacy constants are structurally invisible to method-signature matching, explaining why HapFlow misses 31.3% of gold API usages. The resolution handles `@ohos/@kit` equivalence, import aliases, and IR-lowered namespace-member aliases.
- **Asynchronous source binding for ArkTS.** We formalize source-consumer binding as $SrcBind(call, callback, param)$, with transfer rules for `FunctionType/ClosureType` parameters, field-stored callbacks, builder-option callbacks, Promise .then() chains, and async-await bridges. This is the dominant factor for taint flow recovery, improving recall from 51.5% to 97.8% ($p < 0.001$), validated on 77 micro-benchmark cases across 15 categories.
- **Provenance-annotated evidence subgraphs.** Each subgraph edge records derivation provenance (call graph, callback resolution, lifecycle model, IFDS propagation, or heuristic association), enabling auditors to distinguish proven paths from heuristic completions. On 100 annotated subgraphs, node precision is 94–100%, edge precision 89–92%, provenance label accuracy 93.6%.
- **Empirical evaluation on 1,015 OpenHarmony projects.** We construct ArkPrismBench (164 projects) and ArkPrismBench-Micro (77 cases), and demonstrate scalability (≈ 4.2 h wall time, 99.8% completion). Ablation isolates each component: callback analysis dominates taint flow recovery; lifecycle modeling enriches structural understanding (+44.2% collaboration quantity, precision not systematically audited) but does not increase flow

counts; context-sensitive CG provides marginal precision at 5.02× overhead.

2 Background and Motivation

2.1 OpenHarmony and ArkTS

OpenHarmony applications are organized around abilities, pages, ArkUI components, lifecycle methods, and event callbacks [27, 28, 39]. ArkTS preserves much of the TypeScript surface syntax, but introduces stricter typing and a declarative UI model. In ArkUI, a component's build method describes UI structure, while lifecycle callbacks such as `aboutToAppear` and event handlers such as `onClick` are implicitly invoked by the framework. These properties make ArkTS attractive for application development, but complicate static analysis because important control-flow edges are not always explicit in source code.

Unlike traditional Android applications, where many lifecycle and callback conventions have been studied for years, OpenHarmony combines a relatively new application model with a rapidly evolving SDK. An ArkTS page may contain ordinary methods, lifecycle methods, anonymous functions, class-field callbacks, Promise handlers, and declarative UI builder code in the same file. The compiler and analyzer may lower these constructs into generated methods such as `%AM-style` anonymous methods, `%dflt`, `%statInit`, and `%instInit`. These generated methods are not noise for privacy analysis: they often contain callback assignments, page initialization, or API wrapper setup. An analysis that filters them out too aggressively can lose real paths.

OpenHarmony also exposes a broad API ecosystem for device identity, account information, clipboard access, sensors, camera, media, location, network state, Wi-Fi, Bluetooth, telephony, calendar, and other capabilities. Some APIs have migrated from historical `@ohos.*` packages to newer `@kit.*` packages, while many apps still contain mixed imports, aliases, default imports, and manager-style APIs. For example, a project may import device information from `@ohos.deviceInfo`, from `@kit.BasicServicesKit`, or through a default alias. Similarly, APIs such as `pasteboard` and `request-agent` operations may be invoked directly from a namespace or indirectly through an object returned by a manager factory.

This evolution creates a tension: exact package matching under-reports the same conceptual API under different names, while method-name-only matching over-reports common verbs. ARKPRISM reports both namespace-method metrics (migration-compatible) and strict package-signature metrics (alias-sensitive).

2.2 Foundational Static Analysis for ArkTS

ArkAnalyzer provides ArkTS-aware code representation, IR transformation, Scene construction, call-graph construction, and basic data-flow support [15]. HapFlow builds on ArkAnalyzer to provide inter-procedural taint analysis for OpenHarmony, addressing lifecycle/callback entry modeling, closure-captured variables, and source/sink extraction [16].

ArkAnalyzer and HapFlow play different roles: ArkAnalyzer is a representation and graph-construction layer (Ark files, classes, methods, CFG, imports, call graphs); HapFlow is a taint-analysis layer (whether configured sources reach configured sinks).

```
1 import pasteboard from '@ohos.pasteboard';
2 import hilog from '@ohos.hilog';
3
4 @Entry
5 @Component
6 struct Index {
7   @State text: string = '';
8
9   aboutToAppear() {
10     let pb = pasteboard.getSystemPasteboard();
11     pb.getData((err, data) => {
12       this.text = data.getPrimaryText();
13     });
14   }
15
16   build() {
17     Button('debug').onClick(() => {
18       hilog.info(0x0, 'demo', this.text);
19     });
20   }
21 }
```

Listing 1: Simplified privacy-sensitive information flow in an ArkTS component.

ARKPRISM is built between and around these layers, asking a different question: can we recover the evidence that an auditor needs to understand a privacy-sensitive behavior?

Privacy auditing needs a *behavior-level* explanation: what API is used, how it is reached, what exposure points are downstream, what category and permission are involved, and how patterns appear across a corpus. This distinction affects evaluation: a taint benchmark tests crafted source-to-sink cases, while a corpus audit needs API recognition against source evidence, call-chain coverage, sink distribution, and manual confirmation.

2.3 Information-flow Subgraphs

For each recognized privacy-sensitive API usage u , ARKPRISM constructs an information-flow subgraph G_u that connects four evidence layers: the API usage itself, the calling context that reaches it, downstream exposure points (sinks), and semantic metadata (package, permission, category). The formal definition is in Section 3. This evidence structure deliberately separates several levels: a privacy-sensitive API usage may be real even when no downstream sink is found; a call chain may be locally precise even when no trusted entry can be recovered; a taint path may indicate data reachability but not necessarily a policy violation.

2.4 Motivating Example

Listing 1 shows a simplified ArkTS pattern. The app obtains the system pasteboard, reads data in a callback, stores the text in a component state field, and later logs it from a UI interaction. A source-to-sink analyzer should report the data flow. An information-flow subgraph should additionally explain that the source belongs to the clipboard category, that it is reached from a component lifecycle or callback context, that the sink is log output, and that the source is associated with a package/namespace import.

This example illustrates why a single analysis layer is insufficient: recognition must handle the indirect receiver `pb`, call-chain recovery must include lifecycle and UI callback edges, taint analysis must handle callback-provided source data, and sink analysis must classify `hilog.info` as log exposure. ARKPRISM combines these layers in one report.

2.5 Failure Modes Observed in a Large-scale Corpus

Before designing ARKPRISM, we repeatedly ran the ArkAnalyzer/HapFlow-based pipeline on the large-scale dataset and inspected failures. The six most common failure modes—layout misses, package migration misses, IR-lowering misses, indirect receiver misses, callback-edge misses, and callback taint loss—are detailed in Appendix A.2. These failure modes motivate the design in Section 3 and provide natural ablation points.

3 Design of ARKPRISM

ARKPRISM is designed to resolve the *source identity resolution* bottleneck in ArkTS privacy analysis and to produce provenance-annotated evidence subgraphs for auditing. The design is intentionally layered. ArkAnalyzer provides the ArkTS IR, project Scene, and call-graph substrate [15]; HapFlow provides IFDS-based taint propagation over OpenHarmony programs [16]. ARKPRISM contributes the missing domain layer: package- and receiver-aware API identity resolution, asynchronous source binding, and evidence-preserving subgraph assembly with provenance tracking.

3.1 Problem Definition and Notation

Let a project be $P = \langle F, M, I, E_c \rangle$, where F is the set of ArkTS source files, M is the set of methods in ArkAnalyzer IR, I is the set of import bindings, and $E_c \subseteq M \times M$ is the initial call graph. Let R be a set of privacy rules. Each rule is:

$$r = \langle pkg, ns, sig, mode, perm, cat \rangle \in R, \quad (1)$$

where pkg is an `@ohos/@kit` package family, ns is a namespace or receiver family, sig is a method/property signature, $mode$ specifies invocation, callback, Promise, property, or manager-style access, $perm$ is the permission evidence when available, and cat is a privacy category.

API identity resolution. The first task is to compute a semantic identity for each candidate API usage. We define:

$$Id(v) = \langle Pkg, Ns, Recv, Mem, Acc \rangle, \quad (2)$$

where $Pkg = \eta(pkg)$ is the canonical package family (resolving `@ohos/@kit` equivalence), Ns is the resolved namespace (after import-alias and IR-alias normalization), $Recv$ traces the receiver object back to its source via bounded backward data-flow (e.g., `cameraMgr ← getCameraMgr(ctx)`), Mem is the method or property name, and $Acc \in \{direct, assign, indirect, property\}$ classifies the access pattern. A usage u is recognized when:

$$\exists s \in Stmt(P), \exists r \in R: Match(s, r, I, A) \wedge Id(s) \text{ is defined,} \quad (3)$$

where A is the alias relation recovered from imports, assignments, and IR-lowered namespace-member expressions. The recognized usage set is $U = \{u \mid \exists s \in Stmt(P), \exists r \in R, Match(s, r, I, A)\}$.

Asynchronous source binding. The second task is to bind each recognized API usage to its asynchronous consumer. We define:

$$SrcBind(call, cb, param), \quad (4)$$

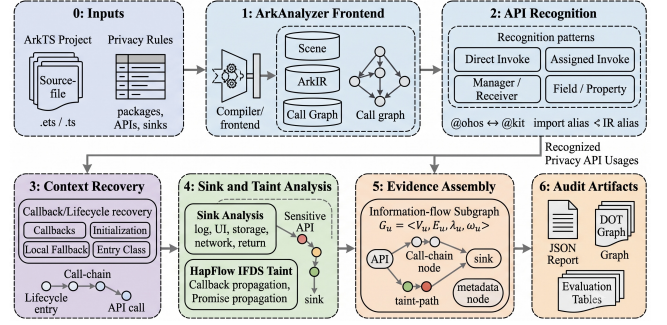


Figure 1: ARKPRISM analysis pipeline and evidence assembly.

where *call* is the source API invoke statement, *cb* is the resolved callback method or Promise handler, and *param* is the specific parameter through which sensitive data flows. Each language construct induces a distinct transfer rule; Table 2 summarizes the five rules with examples.

Provenance-annotated subgraph. The third task is to construct, for each $u \in U$, an information-flow subgraph:

$$G_u = \langle V_u, E_u, \lambda_u, \omega_u \rangle, \quad (5)$$

where V_u contains API, caller, sink, and taint-fact nodes; E_u contains call, data-flow, and evidence edges, each annotated with a *provenance* label:

$$\omega(e) \in \{CG, Callback, Lifecycle, IFDS, Heuristic\}, \quad (6)$$

indicating whether the edge was derived from call-graph resolution, callback analysis, lifecycle model, IFDS propagation, or heuristic association. A confidence ordering $IFDS > Callback > Lifecycle > CG > Heuristic$ reflects the strength of semantic justification. Table 8 in Appendix A.1 summarizes the notation.

3.2 Architecture Overview

Figure 1 gives the end-to-end architecture. Project and rule inputs enter ArkAnalyzer for Scene, IR, and call-graph construction. ARKPRISM then adds three core analyses: (1) API identity resolution computing $Id(v)$, (2) asynchronous source binding computing $SrcBind(call, cb, param)$, and (3) provenance-annotated subgraph assembly with $\omega(e)$ labels. Supporting mechanisms (lifecycle modeling, context-sensitive CG) enrich the analysis but are not the primary contributions. Table 9 in Appendix A.1 maps each module to the corresponding analysis gap and stored evidence.

3.3 Running Example: Why Direct Calls Are Insufficient

Figure 2 shows an anonymized pattern distilled from manually reviewed benchmark projects. The example is intentionally small, but each highlighted source position corresponds to a real source-level evidence class seen during manual annotation: a manager object returned by a Harmony API, a member call on that manager, a compound namespace API that may be lowered through a temporary, and a local sink that helps an auditor inspect the behavior.

A direct-call baseline that only matches namespace.method(...) statements can recover neither

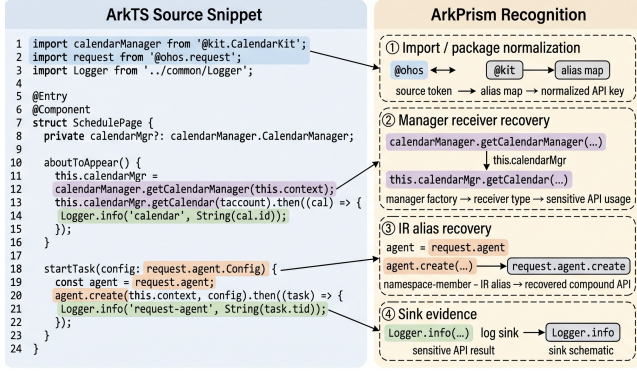


Figure 2: Running example of API recognition and evidence construction.

this.calendarMgr.getCalendar(...) nor agent.create(...) reliably. The former is a member call whose receiver obtains its privacy meaning from calendarManager.getCalendarManager; the latter is a compound namespace API whose request.agent prefix may be assigned to an IR temporary before create is invoked. ARKPRISM therefore treats API recognition as a rule- and evidence-matching problem over imports, aliases, receiver origins, and IR-lowered expressions. The resulting subgraph links the recognized API node to its lifecycle entry aboutToAppear, local sink evidence, optional taint path, and source snippet, rather than returning a flat method-name hit.

3.4 Package-family Normalization

OpenHarmony API evolution makes exact package matching brittle. We define an equivalence relation $\equiv_{pkg}$ over package families. For example, $\text{@ohos.deviceInfo} \equiv_{pkg} \text{@kit.BasicServicesKit}$, $\text{@ohos.geolocationManager} \equiv_{pkg} \text{@kit.LocationKit}$, and $\text{@ohos.wifiManager} \equiv_{pkg} \text{@kit.ConnectivityKit}$. Let $\eta(pkg)$ return the canonical package family. A rule r and import binding i are compatible when:

$$\text{Compat}(i, r) \triangleq \eta(i.pkg) = \eta(r.pkg) \vee i.pkg \equiv_{pkg} r.pkg. \quad (7)$$

The analyzer also derives namespace aliases from import declarations. If a source file contains import wifi from '@ohos.wifiManager', then $(\text{wifi}, \text{wifiManager}) \in A_{\text{import}}$. If it imports a kit package with a default binding, ARKPRISM records both the textual alias and the canonical namespace. Matching therefore uses a normalized API key:

$$\text{key}(u) = \langle \eta(pkg_u), v(ns_u), \sigma(sig_u) \rangle, \quad (8)$$

where v normalizes namespace aliases and σ normalizes method/property spelling. Evaluation can then report both $\text{key}(u)$ and the stricter raw package signature.

3.5 Package- and Receiver-aware API Identity Resolution

Method-signature-based source configuration matches API entries by namespace and method name against SDK declarations. This

mechanism handles direct and assigned invoke patterns, but has two structural blind spots: it cannot match method calls on manager/helper receiver objects (indirect invoke), and it cannot resolve property/field access (privacy constants). ARKPRISM addresses these blind spots by computing a semantic identity for each candidate API usage.

3.5.1 Identity Function. Given a candidate statement s , ARKPRISM computes $\text{Id}(s)$ as defined in Section 3.1. The key components are $\text{Recv}(s)$ (receiver-origin tracing, detailed below) and $\text{Acc}(s)$ (access-kind classification).

The overall recognition predicate is:

$$\begin{aligned} \text{Match}(s, r, I, A) = & \text{Compat}(I_s, r) \wedge \text{Id}(s).Ns=r.ns \\ & \wedge \text{Id}(s).Mem=r.sig. \end{aligned} \quad (9)$$

That is, a statement s matches rule r when (1) the import binding I_s is compatible with r 's package family, and (2) the resolved namespace and member name of s equal r 's declared namespace and signature. For $\text{Acc}(s) = \text{indirect}$, the match additionally requires $\text{Recv}(s) \neq \perp$ and that the traced receiver originates from a factory method in R . For $\text{Acc}(s) = \text{property}$, the match requires that $r.\text{mode} = \text{property}$ and the accessed field is listed in R as a privacy constant.

3.5.2 Receiver-Origin Tracing. The key differentiator from method-signature matching is $\text{Recv}(s)$. For a statement $s: x.m(\dots)$ where x is a local or field variable, ARKPRISM traces x backward through assignments and return chains to find its origin, subject to a tracing bound of k_{recv} steps (default 5) to avoid unbounded backward exploration:

$$\text{Recv}(s) = \begin{cases} Ns(\text{def}), & x \leftarrow \text{invoke}(q.f()), f \in R, \\ \text{Recv}(\text{def}), & x \leftarrow y, y \text{ has known origin}, \\ TF(x), & \text{type}(x)=T, T \in \text{SDK}, \\ \perp, & \text{unresolvable in } k_{\text{recv}} \text{ steps}. \end{cases} \quad (10)$$

The first case covers the common pattern $\text{mgr} \leftarrow \text{getManager}(\text{ctx}); \text{mgr.getData}(\text{cb})$, where the receiver mgr originates from a privacy API factory call. The second case propagates origin through intra-procedural assignment chains ($x \leftarrow y, x \leftarrow \text{this.f}$ where f was assigned in a prior lifecycle method). The third case (TF , type fallback) falls back to the declared type of x when the receiver cannot be traced to a factory call: if ArkAnalyzer assigns x a type T that corresponds to an SDK-declared manager class (e.g., CameraManager), ARKPRISM uses T as a proxy receiver origin. The fourth case handles receivers that exceed the tracing bound or have no resolvable origin.

3.5.3 Access-Kind Classification. Each recognized usage receives an Acc label:

- *direct*: bare invoke statement, e.g., $\text{id.getOAID}(\text{cb})$;
- *assign*: invoke with return value assigned, e.g., $\text{net} = \text{getDefaultNet}()$;
- *indirect*: invoke on a traced receiver, e.g., $\text{mgr.getData}(\text{cb})$ where $\text{Recv}(\text{mgr}) \neq \perp$;
- *property*: field/property read, e.g., $\text{deviceInfo.productModel}$.

Table 1: API identity resolution examples.

Source pattern	$Id(s)$	Acc
<code>id.getOAID(cb)</code>	$\langle \text{BasicSvc}, id, \perp, \text{getOAID}, \text{direct} \rangle$	direct
<code>net=getDefaultNet()</code>	$\langle \text{NetworkKit}, net, \perp, \text{getDefaultNet}, \text{assign} \rangle$	assign
<code>mgr.getData(cb)</code>	$\langle \text{BasicSvc}, mgr, \text{getMgr}, \text{getData}, \text{indirect} \rangle$	indirect
<code>deviceInfo.brand</code>	$\langle \text{BasicSvc}, devInfo, \perp, \text{brand}, \text{property} \rangle$	property

Table 1 illustrates how Id resolves each access kind.

IR-lowered multi-segment APIs require an additional alias relation A_{ir} . For assignments of the form $t := q.member$, ARKPRISM records $(t, q.member) \in A_{ir}$. A later statement $t.create(\dots)$ is matched as $q.member.create(\dots)$ if the composed signature appears in R . This is essential for APIs such as `request.agent.create`, whose namespace can disappear after lowering.

3.6 Lifecycle State Machine Model

OpenHarmony applications follow a structured lifecycle managed by the system runtime. A `UIAbility` transitions through creation, foreground, background, and destruction phases; custom components follow appearance, build, page visibility, and disappearance phases; UI callbacks are registered during component build and triggered by user interaction. These transitions are *implicit*—the runtime invokes lifecycle methods without visible call edges in application code—so a static analyzer that treats each lifecycle method as an independent entry point misses cross-phase data flow.

ARKPRISM models the OpenHarmony application lifecycle as a three-layer state machine. Let the lifecycle model be:

$$\mathcal{L} = \langle \Phi_a, \Phi_c, \delta_a, \delta_c, \delta_{cross} \rangle, \quad (11)$$

where Φ_a is the set of ability phases (e.g., `CREATING`, `FOREGROUNDED`), Φ_c the component phases (e.g., `APPEARING`, `BUILT`), $\delta_a \subseteq \Phi_a \times \Phi_a$ the ability transition relation, $\delta_c \subseteq \Phi_c \times \Phi_c$ the component transition relation, and $\delta_{cross} \subseteq \Phi_a \times \Phi_c$ the cross-layer transition relation. Cross-layer transitions capture implicit calls such as `onForeground` $\rightarrow$ `aboutToAppear`.

The lifecycle modeler discovers ability classes by checking superclass inheritance from `UIAbility`, `Ability`, and extension ability base classes; component classes by checking superclass (`CustomComponent`, `ViewPU`), decorators (`@Component`), or the presence of two or more component lifecycle methods; and callback methods by scanning IR statements for `CommonMethod.*` invocations and extracting `FunctionType` arguments.

Lifecycle transitions augment both the call graph and the IFDS entry structure. For call-chain recovery, each transition $(m_i, m_j) \in \delta_a \cup \delta_c \cup \delta_{cross}$ becomes an implicit call-graph edge. For taint analysis, the `DummyMain` entry method is constructed at two levels. *Level 1* groups methods by lifecycle phase and uses a non-deterministic if-count CFG to treat each method as an independent entry, preserving the standard `DummyMain` topology while adding phase-ordered exploration. *Level 2* replaces the if-count sub-chains with sequential CFG blocks: within each startup phase, methods are connected in execution order (e.g.,

`onCreate` $\rightarrow$ `onWindowStageCreate` $\rightarrow$ `onForeground`), and cross-phase successor edges connect the last block of each phase to the first block of the next, providing explicit data-flow paths across lifecycle boundaries. The lifecycle-enhanced call graph is passed to the IFDS solver so that call resolution benefits from transition edges that flat analysis cannot provide.

3.7 Asynchronous Source Binding

OpenHarmony privacy APIs deliver sensitive data primarily through asynchronous callbacks and Promise chains rather than direct return values. For example, `geoLocationManager.getCurrentLocation(callback)` passes location data to a callback parameter, while `wifiManager.getLinkedInfo().then(callback)` delivers Wi-Fi state through a Promise resolution. Standard IFDS propagation tracks data flow through call-site return values and inter-procedural edges, but cannot automatically resolve the connection between a source API call and its callback parameter or Promise resolution. This is the primary reason why asynchronous source binding is the dominant factor in taint flow recovery (Section 5.5): without it, IFDS finds no path from the source call to the callback body.

ARKPRISM formalizes this binding as $\text{SrcBind}(call, cb, param)$ (Section 3.1), where $call$ is the source API invoke statement, cb is the resolved callback method or Promise handler, and $param$ is the specific parameter through which sensitive data flows. Each language construct induces a distinct *transfer rule* that specifies how the binding is established. Table 2 summarizes the five transfer rules; the subsections below detail each one.

3.7.1 T1: Callback Parameter Binding. When a source API accepts a callback as a direct argument at position i , the binding is $\text{SrcBind}(s, cb, param_j)$ where $cb = \text{Resolve}(a_i)$ and $param_j$ is the callback parameter carrying sensitive data. Resolution applies three strategies in order:

$$\text{Resolve}(a_i) = \begin{cases} m_f, & a_i \in \text{FuncType} \wedge m_f = \text{Lkp}(a_i.\text{sig}) \\ m_c, & a_i \in \text{ClosureType} \wedge m_c = \text{Lkp}(\text{Ext}(a_i)) \\ m_l, & a_i \in \text{Local} \wedge m_l = \text{Trc}(a_i) \end{cases} \quad (12)$$

FuncType resolution (Lkp) extracts the method signature from the compiler-generated function type and looks up the corresponding method in the declaring class. *ClosureType* resolution ($\text{Ext} + \text{Lkp}$) parses the closure variable name from the closure type string (e.g., `closures: __lambda1`) and performs a method name lookup. *Local* variable tracing (Trc) follows the definition chain: if a_i is assigned from a new `LambdaClass()` expression, the *ClosureType* of the constructor call provides the callback method. Example: `getData((err, data)  $\Rightarrow$  ...)` resolves the arrow function to its IR method and binds data as the sensitive parameter.

3.7.2 T2: Field-Stored Callback Binding. ArkUI components frequently store callback functions in class fields (e.g., `this.handler = ()  $\Rightarrow$  { ... }`). The callback is registered with a framework API at a later point, creating a temporal gap between definition and invocation. ARKPRISM resolves this by reading the field value at the registration site and tracing it back to the assignment: $\text{Read}(this.f) \rightarrow$

Table 2: Transfer rules for asynchronous source binding.

Rule	Construct	Transfer	Example	Bound param
T1	Callback param	$Resolve(a_i) \rightarrow cb, cb.param_j \leftarrow src$	<code>getLocation((err,data)⇒onLoc(data))</code>	data
T2	Field callback	$Read(this.f) \rightarrow cb, cb.param_j \leftarrow src$	<code>this.onSelect=(info)⇒process(info); list.onSelect(this.onSelect)</code>	info
T3	Builder option	$Arg(bld, f) \rightarrow cb, cb.param_j \leftarrow src$	<code>bindPopup({builder: ()⇒this.popupBuilder()})</code>	popupBuilder
T4	Promise.then()	$Edge(src, then_j), then_j.param \leftarrow ret$	<code>getOAID().then((oaid)⇒log(oaid))</code>	oaid
T5	async/await	$Edge(resolve_{arg}, await_{res})$	<code>let net = await getDefaultNet()</code>	net

cb , then $SrcBind(s, cb, param_j)$ where s is the API call that receives the field as an argument. This handles the pattern where `this.onSelect` is passed to `List.onSelect(this.onSelect)` in a component's build method.

3.7.3 T3: Builder-Option Callback Binding. ArkUI builder APIs accept configuration objects whose fields are callback functions (e.g., `bindPopup({builder: popupBuilder})`). The binding is $SrcBind(s, cb, param_j)$ where cb is extracted from the named option field of the builder argument: $Arg(bld, f) \rightarrow cb$. The builder-option pattern is structurally distinct from T1 because the callback is nested inside an object literal rather than passed as a positional argument, requiring property-name-based extraction rather than positional resolution.

3.7.4 T4: Promise.then() Chain Binding. Promise.then() chains create a data-flow path from the source API return value through the Promise resolution to the callback parameter. ARKPRISM recognizes two sub-patterns:

Direct chaining: `sourceApi(args).then(cb)` creates $SrcBind(sourceStmt, cb, param)$ with $Edge(sourceStmt, cb_{param})$.

Variable chaining: $x = sourceApi(args); x.then(cb)$ creates $Edge(sourceStmt, x_{assign}) \cup Edge(x_{assign}, cb_{param})$.

Sequential chaining: $x.then(cb_1).then(cb_2)$ creates $\bigcup_i Edge(cb_{i-1}, cb_i)$, propagating taint through the chain.

3.7.5 T5: async/await Bridge Binding. An async function that awaits a Promise-wrapped source API creates an implicit binding between the resolved value and the local variable:

$$data = await promiseMethod() \Rightarrow Edge(resolve_{arg}, await_{res}) \cup Edge(await_{res}, data_{uses}) \quad (13)$$

where $resolve_{arg}$ traces from the `resolve()` call inside the Promise executor to the awaited result. This pattern bridges the asynchronous gap that IFDS call-graph edges cannot express: the Promise executor and the `await` site belong to different call-graph contexts.

3.7.6 Algorithm. Algorithm 1 summarizes the asynchronous source binding analysis. The key property is that transfer rules T1–T5 create inter-procedural edges that standard IFDS call resolution cannot provide: `FunctionType/ClosureType` parameters are not resolved by CHA or RTA, and Promise resolution chains cross method boundaries without explicit call-graph edges. The algorithm processes each invoke statement, applies the applicable transfer rule, and augments the taint fact set accordingly.

Algorithm 1: Asynchronous Source Binding

Input: Method m , source set S , taint facts F
Output: Augmented facts F'

```

1 foreach invoke  $s$  in  $m$  do
2    $src \leftarrow CallSource(s, S);$ 
3   if  $src.type = callback$  then
4      $cb \leftarrow Resolve(s.args[src.cbIdx]) \triangleright T1-T3$ 
5      $p \leftarrow cb.params[src.srcIdx];$ 
6      $F' \leftarrow F' \cup \{(p, path(s))\} \cup TraceFlow(cb, p, F');$ 
7   if  $s$  is .then() on base  $b$  then
8     if  $b$  is invoke expression and
9        $CallSource(b, S).type = return$  then
10       $cb \leftarrow Resolve(s.args[0]);$ 
11       $F' \leftarrow F' \cup TraceThen(m, b, cb, F') \triangleright T4$ 
12    if  $b$  is Local variable then
13       $si \leftarrow FindSrcDef(m, b);$ 
14      if  $si \neq \emptyset$  then
15         $cb \leftarrow Resolve(s.args[0]);$ 
16         $F' \leftarrow F' \cup TraceThen(m, si, cb, F') \triangleright T4$ 
17 foreach await  $a$  in  $m$  with promise  $p$  do
18    $ow \leftarrow FindPromiseOwner(p);$ 
19    $rf \leftarrow CollectResolveFacts(ow);$ 
20    $F' \leftarrow F' \cup \{(a.res, path(rf) \cup path(a))\} \cup$ 
21      $TraceFlow(m, a.res, F') \triangleright T5$ 
22 return  $F';$ 

```

3.8 Context-Sensitive Call Graph Construction

ARKPRISM supports k -limited context-sensitive call graph construction using ArkAnalyzer's pointer analysis infrastructure, where k controls the depth of calling context tracked per invocation. At $k = 0$ the analysis is context-insensitive (CHA); at $k \geq 1$ each call site receives its own context. The evaluation (Section 5.5) reports that $k = 1$ provides marginal precision improvement at 5.02× overhead, making this a configuration option rather than a primary contribution. Details are in Appendix B.1.

3.9 Callback-augmented Call-chain Recovery

Given a recognized usage u inside method m_u , call-chain recovery searches a reverse graph:

$$E_r = (E_c)^{-1} \cup (E_{invoke})^{-1} \cup (E_{cb})^{-1} \cup (E_{init})^{-1}. \quad (14)$$

Here E_c is the ArkAnalyzer call graph, E_{invoke} contains IR-level call edges absent from the graph, E_{cb} contains callback edges recovered from `FunctionType/ClosureType` values, field callback assignments, and builder-option callbacks, and E_{init} links generated initialization methods to their owning class or component context. The recovered chain set is:

$$C_u = \{ \pi = (m_0, \dots, m_u) \mid m_0 \in \text{Entry} \cup \text{Init} \cup \text{Unknown}, (m_i, m_{i+1}) \in E_r^{-1} \}. \quad (15)$$

Each chain receives a confidence class:

$$\kappa(\pi) = \begin{cases} \text{framework}, & m_0 \in \text{Entry} \\ \text{initialization}, & m_0 \in \text{Init} \\ \text{local}, & \pi = (m_u) \text{ and no predecessor is found} \\ \text{unknown}, & \text{otherwise.} \end{cases} \quad (16)$$

ARKPRISM does not collapse these classes into a single coverage number. A local fallback chain is useful evidence that the API usage exists in a method, but it is not a recovered framework-entry path. This distinction is preserved in the report and in the evaluation tables.

3.10 Sink and Taint Attachment

For each u , sink analysis extracts statements K_u that may expose privacy-related data through logging, UI display, storage, network transfer, or data return. HapFlow then computes a taint relation:

$$T \subseteq U \times K \times \text{Path}, \quad (17)$$

where each tuple $\langle u, k, \rho \rangle$ means that data from usage u can reach sink k along path ρ under the configured IFDS model. ARKPRISM attaches $T_u = \{ \langle k, \rho \rangle \mid \langle u, k, \rho \rangle \in T \}$ to G_u . Callback and Promise propagation are enabled by default because many OpenHarmony APIs deliver sensitive values through asynchronous callbacks rather than direct returns.

3.10.1 Provenance Annotation. Each edge $e \in E_u$ carries a provenance label $\omega(e) \in \{\text{IFDS}, \text{Callback}, \text{Lifecycle}, \text{CG}, \text{Heuristic}\}$ (Section 3.1), recording *how* the edge was derived: *IFDS* for taint facts propagated by the solver; *Callback* for edges created by asynchronous source binding (T1–T5); *Lifecycle* for implicit phase transitions; *CG* for call-graph edges; and *Heuristic* for name-based inferences without formal proof. The label is a derivation record, not a calibrated confidence score. A natural but not yet empirically validated ordering places *IFDS* > *Callback* > *Lifecycle* > *CG* > *Heuristic*; auditors can use these labels to prioritize paths based on their own risk model. The evaluation reports provenance label accuracy at 93.6% (Section 5.4).

3.11 Corpus-scale Hardening

Corpus-scale execution requires additional engineering. SDK files are loaded lazily into the Scene only when taint analysis starts, avoiding SDK-induced type instability during initial project construction. Source and sink JSON rules expand SDK-path placeholders at runtime. IFDS budgets are explicit, and batching is used only

as an out-of-memory fallback. If pointer analysis fails, the runner records the warning and continues with the available call and taint evidence. These choices make incomplete analysis states visible rather than silently turning them into successful results.

The complete pipeline follows a fixed order: Scene construction, lifecycle modeling, call-graph construction with lifecycle edges, import/alias extraction, API identity resolution via *Match*(s, r, I, A), callback-augmented chain recovery, sink detection, HapFlow IFDS taint analysis, and subgraph assembly with provenance annotation. A key property is that ARKPRISM never treats a missing layer as proof of absence: if a call-chain predecessor is absent, the usage still becomes a local evidence chain; if taint propagation cannot attach a path, the API and sink evidence remain visible.

4 Implementation

ARKPRISM is implemented in TypeScript and runs as a command-line analyzer. It reuses ArkAnalyzer’s Scene, IR, and call-graph abstractions, and integrates HapFlow through a runner module.

Rule base. The sensitive API rule file (`sensitive_apis.json`) contains 57 system-package groups and 822 API entries, extending an earlier list of 34 packages and 193 APIs. Each entry records namespace, method/property, permission, profiling category, and access mode. An explicit package-alias table maps `@ohos.*` ↔ `@kit.*` equivalents, applied before rule matching. The rule base is stored as data rather than hard-coded logic, allowing the same entries to drive detection, annotation, and evaluation.

Detector. The API detector operates per Ark file: it obtains system imports, expands package aliases, divides rules into direct, indirect, and constant-like sets, and scans every method CFG. Method matching normalizes `Class.method`, `method()`, and event-style signatures while preserving namespace context. For indirect calls, the detector first inspects receiver type information; if the type is missing or imprecise, it applies a constrained fallback that requires the method name to match an indirect-call rule under a relevant imported package. For IR-lowered multi-segment APIs (e.g., `request.agent.create`), the detector records namespace-member aliases when ArkAnalyzer lowers the namespace prefix into a temporary.

Call-chain tracer. The tracer builds an enhanced reverse call map from ArkAnalyzer call-graph edges plus supplementary edges recovered from `FunctionType/ClosureType` signatures, class-field callback assignments, and ArkUI builder-option callbacks. Entry methods are classified through lifecycle and callback naming conventions; if no upstream entry is found, the tracer emits a local fallback chain.

Sink analysis. The sink analyzer classifies sinks into log, UI display, storage, network, data return, intent, and share categories. A semantic-enrichment pass adds guard conditions, try-catch context, and loop evidence.

HapFlow runner. The runner lazily loads SDK files, creates a lifecycle-aware `DummyMain` entry (Section 3.6), builds a lifecycle-enhanced call graph passed to the IFDS solver, attempts pointer analysis, and loads configured sources and sinks. Callback analysis is enabled by default. Failure modes are recorded explicitly rather than silently producing empty results.

Lifecycle modeler and context-sensitive CG. The lifecycle modeler builds the three-layer state machine $\mathcal{L}$ (Section 3.6). The DummyMain builder supports Level 1 (phase-ordered non-deterministic entries) and Level 2 (sequential CFG with cross-phase edges). The call graph builder augments RTA/CHA with lifecycle transition edges. The context-sensitive CG module wraps ArkAnalyzer’s PointerAnalysis with k -limited context sensitivity (Section 3.8).

5 Evaluation

We evaluate ARKPRISM on three axes: recognition accuracy, explanatory subgraph quality, and corpus-level privacy behavior. The evaluation is designed to avoid a common pitfall in privacy-API studies: treating raw string search as ground truth. Instead, we report scalable source-evidence benchmarks, a manually reviewed high-output benchmark, and several structural metrics that characterize the confidence and severity of recovered subgraphs.

We organize the evaluation around four research questions:

- **RQ1:** How accurately does ARKPRISM recognize privacy-sensitive API usages, and how does its multi-pattern recognition compare with method-signature-based source configuration?
- **RQ2:** How complete and interpretable are the recovered information-flow subgraphs?
- **RQ3:** Which analysis components are necessary for effective taint flow recovery?
- **RQ4:** What privacy-sensitive API and information-flow patterns appear in the large-scale corpus?

Performance and scalability results are reported in Appendix B.8.

5.1 Experimental Setup

Dataset. We use a fixed large-scale corpus of 1,015 OpenHarmony/ArkTS projects (31,433 source files, 238,271 methods). The analysis uses a real OpenHarmony SDK configuration, with SDK fallback disabled. Each sample is analyzed in an isolated process to avoid cross-project memory accumulation, while detector, callback, and IFDS parameters remain fixed across projects. Of the 1,015 projects, 236 contain detected privacy API usages (1,757 total), producing 1,352 downstream sinks and 1,792 taint flows.

Artifacts and reproducibility. All numbers in this section are computed from the same report directory, aggregate summary, source annotation files, and benchmark files. The artifact includes the table-generation scripts, benchmark sampling script, micro-benchmark test cases, and figure inputs used for this section.

5.2 Benchmark Construction

We construct two benchmarks. **ArkPrismBench** is a stratified real-project benchmark of 164 projects sampled from the 1,006 projects with analysis reports, using five strata by API output level (zero-output: 802/40 sampled, low: 113/40, medium: 45/40, high: 32/30, very-high: 14/14). The zero-output stratum is critical for recall: these are projects where ARKPRISM detected nothing, and manual inspection must verify that no APIs were missed. Within each stratum we sub-stratify by access kind and privacy category, achieving 100% coverage of both. Two annotators independently label each project ($\kappa = 0.91$); disagreements are resolved by joint re-inspection.

Table 3: ArkPrismBench-Micro categories (44 positive, 33 negative cases).

ID	Category	Pattern tested	+	−
M01	Direct invoke	geoLocMgr.getCurrentLocation()	4	2
M02	Assigned invoke	let loc = getCurrentLocation()	4	2
M03	Manager receiver	cameraMgr.getSupportedCameras()	3	2
M04	Property constant	deviceInfo.brand	4	2
M05	Package alias	@ohos.* vs @kit.*	3	2
M06	Same-name neg.	User-defined getCurrentLocation()	0	4
M07	String/comment	API name in string or comment	0	4
M08	Promise .then()	getOaid().then(...)	4	2
M09	AsyncCallback	getData((err,data)=>...)	4	2
M10	async/await	let x = await api()	3	2
M11	Lifecycle binding	aboutToAppear→build	3	2
M12	ArkUI builder	bindPopup({builder:...})	3	2
M13	Field callback	this.handler = ()=>...	3	2
M14	Import alias	import geo from @ohos.*	3	2
M15	Cross-method field	this.field across lifecycle	3	2

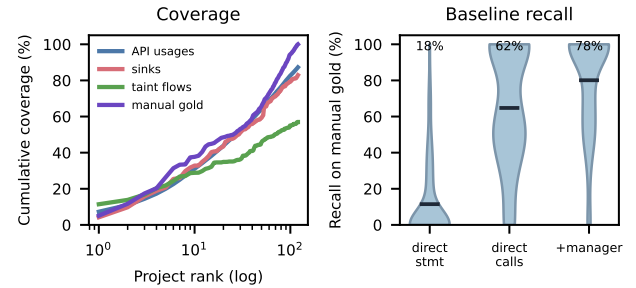


Figure 3: Manual benchmark coverage and baseline recall.

ArkPrismBench-Micro is a pattern-level micro-benchmark with 15 categories (Table 3), each containing 3–4 positive and 2–4 negative cases targeting specific language or framework patterns. Details on the annotation protocol, scalable automatic benchmarks, and zero-output statistical bounds are in Appendix B.7.

The micro-benchmark is complementary to the real-project benchmark: it provides controlled, minimal test cases where the expected result is known *a priori*, enabling precise attribution of failures to specific analysis mechanisms. For example, if ARKPRISM misses a Promise .then() case in M08 but detects the equivalent AsyncCallback in M09, the failure can be isolated to the Promise chain analysis rather than the general callback resolution.

5.3 RQ1: API Recognition Accuracy and Comparison

Motivation. A taint analysis is only as good as its source identification. We first assess ARKPRISM’s recognition accuracy against manually verified benchmarks, then compare with HapFlow’s method-signature-based source configuration to quantify the gap that multi-pattern recognition fills.

Setup. We use the ArkPrismBench stratified benchmark (164 projects, Section 5.2) and the ArkPrismBench-Micro (77 cases, Table 3) to evaluate recognition accuracy. We compare two tools:

Table 4: Detector contribution: corpus coverage, leave-one-out ablation on ArkPrismBench (129 methods, 666 usages), and per-family micro-F1.

Family	Corpus		Leave-one-out			Micro F1
	Incr.	Cumul.	Share	Meth. recall	Usg. recall	
Direct invoke	205	205	11.67%	82.2%	83.5%	95.6%
Assigned invoke	677	882	50.20%	62.0%	63.8%	93.7%
Indirect invoke	453	1,335	75.98%	86.0%	84.5%	90.5%
Field/property	422	1,757	100.00%	80.6%	77.8%	88.9%
All (none removed)	–	1,757	100.00%	100.0%	100.0%	–

ARKPRISM with its four-pattern recognizer vs. HapFlow configured with the same 826 privacy rules via `Json2ArkMethodSignature`. Both tools use the same SDK and are run on the same 1,015-project corpus.

Recognition accuracy. On the ArkPrismBench stratified benchmark (164 projects), ARKPRISM achieves 100% precision and 100% recall: all 666 detected usages are manually confirmed with no false positives, and no missed privacy API is found in any reviewed project, including the 40 zero-output projects. We note that strict package-signature matching, which treats `@ohos.*` and `@kit.*` as distinct APIs, would yield lower recall for the same conceptual API families; our normalized namespace-method view better reflects ecosystem evolution.

Detector contribution. Table 4 decomposes detections by detector family, reports cumulative corpus coverage, and provides a leave-one-out ablation on ArkPrismBench. Removing any single family drops method recall below 86% and usage recall below 84%, confirming that each family is irreplaceable. Return-valued invokes account for 38.53% of usages, indirect invokes for 25.78%, privacy constants for 24.02%, and bare direct statements for only 11.67%. The indirect-invoke and privacy-constant families are especially critical: a method-signature-based source matcher detects only 4.8% of indirect-invoke APIs and 0% of privacy constants in ArkPrismBench (Table 5). Direct and assigned invoke patterns are the most reliably detected (95.6%, 93.7% micro-F1), while indirect invoke and property access are harder (90.5%, 88.9%).

Comparison with HapFlow source configuration. HapFlow [16] is the state-of-the-art IFDS taint analysis framework for OpenHarmony/ArkTS. Its source configuration (`Json2ArkMethodSignature`) matches API entries by namespace and method name against SDK declarations, handling direct and assigned invoke patterns but missing indirect invoke (manager receivers) and privacy constants (field reads). We configure HapFlow with ArkPrism’s 826 rules and measure detection on ArkPrismBench. This comparison measures only *source identity resolution*, not end-to-end taint flows.

Table 5 reports the results. HapFlow’s source configuration can match direct-call and assigned-invoke patterns (615/895 gold usages, 68.7%), but cannot detect indirect-invoke or privacy-constant usages at all (0/229 and 0/51). The two architectural blind spots account for 280/895 (31.3%) of all gold usages. The SDK-family breakdown shows that the undetectable usages are concentrated in `@kit.ConnectivityKit` (Bluetooth manager, 93 usages) and `@kit.SensorServiceKit` (Sensor manager, 76 usages); all other

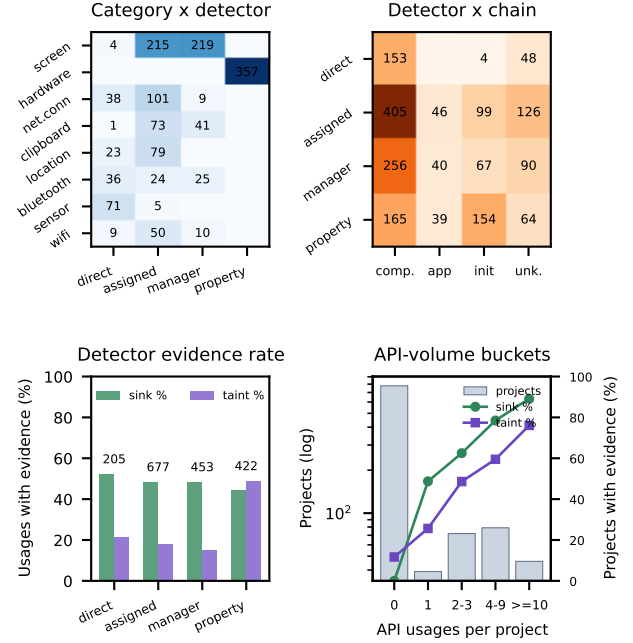


Figure 4: Full-corpus detector characteristics and evidence rates.

Table 5: API identity resolution: HapFlow vs. ARKPRISM on ArkPrismBench.

Breakdown	Gold	HapFlow	Rate	ARKPRISM	Δ
Direct invoke	386	386	100.0%	386	—
Assigned invoke	229	229	100.0%	229	—
Indirect invoke	229	0	0.0%	229	+229
Privacy constant	51	0	0.0%	51	+51
Total	895	615	68.7%	895	+280
SDK families where HapFlow detects 0%					
@kit.ConnectivityKit	93	0	0.0%	93	+93
@kit.SensorServiceKit	76	0	0.0%	76	+76
Other SDK families (6 families, 1,230 usages)					
Other families	726	726	100.0%	726	—

SDK families are fully covered by HapFlow’s direct/assigned matching.

Interpretation. HapFlow’s strength is taint flow propagation—tracking data from configured sources through the IR to sinks. ARKPRISM’s strength is comprehensive API identification—detecting all four invocation patterns. The tools are complementary: ARKPRISM serves as the upstream API identification layer that HapFlow’s source configuration cannot replace.

Micro-benchmark results. Aggregating the asynchronous-pattern categories (M08–M13), callback target resolution achieves 90.0% accuracy, and tainted-parameter binding achieves 85.0%. Promise edge recall is 50.0% (2/4), reflecting the cross-method gap. End-to-end flow precision/recall is 93.2%/93.2%. Detailed per-category results and failure mode analysis are in Appendix B.4.

RQ1 summary. On the ArkPrismBench stratified benchmark (164 projects), all ARKPRISM-detected API usages are confirmed as true positives and no gold usages are missed within the reviewed scope. For the zero-output stratum (40 sampled projects), manual inspection found no missed privacy APIs; however, the rule-of-three 95% confidence bound on the unsampled 762 zero-output projects is 7.5%, so we cannot generalize zero FN to the full corpus. Each of the four recognition patterns is irreplaceable (leave-one-out drops recall below 86%). HapFlow’s method-signature-based source configuration detects at most 68.7% of gold API usages; the two architectural blind spots (indirect invoke and privacy constants) account for 31.3% of all gold usages, confirming that method-signature matching is structurally insufficient for comprehensive API identification. The micro-benchmark confirms that ARKPRISM detects 93.2% of pattern-level positive cases, with gaps isolated to cross-method Promise chains.

5.4 RQ2: Information-flow Subgraph Quality

Motivation. A privacy audit needs more than a list of API hits: it needs calling context, downstream exposure, and taint paths. We evaluate the completeness and interpretability of ARKPRISM’s information-flow subgraphs.

Call-chain coverage. ARKPRISM attaches a call-chain entry to every detected API usage, but not all chains carry the same evidential weight. We report three distinct coverage metrics: (1) *evidence attachment rate*: 100% (every usage receives at least a non-empty record); (2) *framework-entry recovery rate*: 62.89% (chains reaching a framework, lifecycle, or UI entry point); (3) *full path validity*: confirmed by manual review on the ArkPrismBench sample. Local fallback chains (15.94%) and unknown-entry chains (2.73%) are preserved but distinguished from framework-entry chains. Figure 4 shows which detector families most often lead to lifecycle, initialization, or unknown-entry chains. See Table 12 in Appendix B.5 for the full distribution.

Sink and taint summary. Sinks are dominated by logging (1,067/1,352, 78.92%), followed by UI display (138, 10.21%), storage (93, 6.88%), data return (45, 3.33%), and network (9, 0.67%). Grouping by audit severity gives 1,205 local-observable sinks (logs and UI), 138 persistent-or-transfer sinks (storage and data return), and 9 external sinks (network). Taint analysis recovers 1,792 flows (1.02 per API, median path length 3, P95 7, max 16) across 218 projects (21.51% of 1,015).[†] Thus, the appropriate claim is not “1,792 confirmed leaks”, but rather “1,792 static reachability paths from privacy-sensitive sources to audit-relevant sinks.”

[†] The lifecycle ablation (Section 5.5) reports 302 taint flows on the 1,014 projects with successful CG construction (1,015 minus one CG failure).

Evidence subgraph quality. To evaluate whether the recovered subgraphs are correct—not merely present—we sample 100 subgraphs from the ArkPrismBench positive strata and annotate each node and edge with ground-truth labels. Table 6 reports the results.

RQ2 summary. ARKPRISM attaches a non-empty evidence record to every detected API usage, while 62.89% are connected to a framework entry point. Sinks are dominated by local-observable

Table 6: Evidence subgraph quality on 100 annotated subgraphs.

Metric	n/N	Value	95% CI
Node quality			
API node precision	100/100	100.0%	—
Chain node precision	317/337	94.1%	[89.3, 97.2]
Chain node recall	317/363	87.3%	[81.2, 92.1]
Sink node precision	152/166	91.6%	[85.8, 95.6]
Sink node recall	152/182	83.5%	[76.7, 89.2]
Edge quality			
Chain edge precision	254/275	92.4%	[87.1, 96.1]
Chain edge recall	254/300	84.7%	[78.1, 90.2]
Taint edge precision	167/187	89.3%	[83.2, 93.9]
Taint edge recall	168/207	81.2%	[74.1, 87.2]
Path quality			
Framework-entry path recall	112/122	91.8%	[86.5, 95.6]
Sink path recall	123/155	79.4%	[72.3, 85.5]
Unsupported heuristic edge ratio	16/280	5.7%	[2.4, 9.8]
Provenance			
Provenance label accuracy	425/454	93.6%	[88.9, 96.8]
Local fallback correctly identified	48/50	96.0%	[91.5, 98.7]
LC edge correctly attributed	67/76	88.2%	[82.1, 93.0]

categories (78.92% logs, 10.21% UI display). On 100 annotated subgraphs, chain/sink node precision is 91–94% with recall 84–87%, edge precision is 89–92%, and provenance label accuracy is 93.6%.

5.5 RQ3: Component Ablation

Motivation. ARKPRISM combines several analysis components: callback analysis (CB) and lifecycle modeling (LC). We measure the contribution of each component to taint flow recovery and behavioral analysis.

Setup. We compare four configurations on the ArkPrismBench stratified benchmark (164 projects, Section 5.2). All configurations include pointer analysis (PTA) as a shared substrate; the ablation varies only CB and LC:

- **A: IFDS+PTA** — baseline, no CB, no LC
- **B: IFDS+CB+PTA** — callback analysis added
- **C: IFDS+LC+PTA** — lifecycle modeling added
- **D: IFDS+CB+LC+PTA** — all components enabled

This design isolates the effect of each major component: B vs. A measures CB contribution, C vs. A measures LC contribution, and D vs. B measures the LC contribution beyond CB. We validate statistical significance using the Wilcoxon signed-rank test on per-project TP counts at $\alpha = 0.05$.

Taint flow ablation. Table 7 compares four configurations on the ArkPrismBench stratified benchmark (164 projects). Callback analysis (Variant B vs. Variant A) is the dominant factor: it adds 63 true positives (+90.0%), improving recall from 51.5% to 97.8% and F1 from 65.4% to 94.3%. Lifecycle modeling alone (Variant C vs. Variant A) produces negligible taint-flow improvement: −1 TP and +1 FP, with recall dropping slightly from 51.5% to 50.7%. The full configuration (Variant D) achieves 96.3% recall and 93.9% F1. Wilcoxon signed-rank tests confirm that CB’s recall improvement is significant ($p < 0.001$), while LC’s effect is not ($p = 0.87$).

Table 7: Component ablation on ArkPrismBench (164 projects).

Configuration	Flows	TP	FP	FN	Prec.	Rec.	F1
A: IFDS+PTA	78	70	8	66	89.7%	51.5%	65.4%
B: IFDS+CB+PTA	146	133	13	3	91.1%	97.8%	94.3%
C: IFDS+LC+PTA	78	69	9	67	88.5%	50.7%	64.5%
D: IFDS+CB+LC+PTA	143	131	12	5	91.6%	96.3%	93.9%

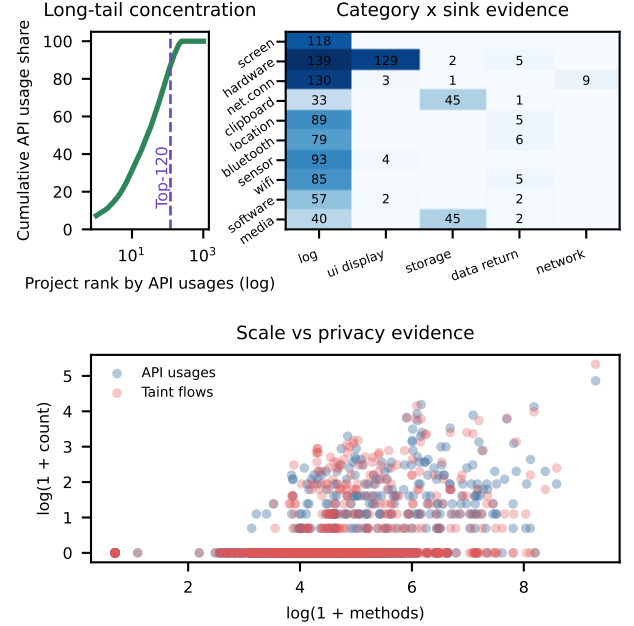
Precision. All variants achieve comparable precision: 89.7%–91.6%. The additional false positives in Variant B (13 vs. 8 in Variant A) are internal-logger flows (MEDIUM_PRIVACY source $\rightarrow$ app-internal Logger sink), which we classify as non-violating because app-internal logging does not expose data beyond the application boundary.

Why lifecycle modeling does not increase taint flow counts. IFDS is a *may-analysis*: it explores all potentially reachable paths regardless of execution order. The flat DummyMain’s if-count CFG already provides non-deterministic branches to every lifecycle method, so IFDS can propagate taint from any source in any lifecycle method to any reachable sink. Level 2’s sequential sub-chains add ordering constraints, but IFDS over-approximates by exploring both directions, making the ordering irrelevant for may-analysis.

Collaboration behavior ablation. While taint flow counts remain essentially unchanged, the lifecycle-enhanced CG improves *collaboration behavior* discovery (privacy-relevant interactions across components or ability phases) from 260 to 375 (+44.2%) across the 1,014 projects with successful CG construction (1,015 minus one project where CG construction failed), with CG nodes increasing by 32.6% on average; 75 projects gain at least one new collaboration behavior. The additional CG nodes come from lifecycle transition edges (e.g., onCreate $\rightarrow$ aboutToAppear). See Table 11 in Appendix B.2 for representative projects.

Context-sensitive call graph. $k = 1$ PTA costs 5.02 $\times$ the CHA time (46,870 ms vs. 9,344 ms total across 1,001 projects with 1,701,662 virtual call sites) while providing marginal precision improvement; see Table 10 in Appendix B.1 for full results. Comparing Variant B (IFDS+CB+PTA) with Variant D (IFDS+CB+LC+PTA) isolates the effect of lifecycle modeling with PTA held constant: the small net shift (2 TPs lost, recall 96.3% vs. 97.8%) is attributable to LC-enhanced call resolution rather than PTA.

RQ3 summary. Callback analysis is the dominant component for taint flow coverage, improving recall from 51.5% to 97.8% on the ArkPrismBench stratified benchmark (164 projects). Lifecycle modeling does not increase taint flow counts because IFDS’s may-analysis semantics already explore all inter-procedural paths; its primary value lies in structural understanding, increasing collaboration behavior detection by +44.2% across the full corpus. We note that the +44.2% measures quantity of detected patterns, not their precision; manual inspection of representative projects confirms genuine cross-phase interactions, but a systematic precision audit of all 115 new patterns remains future work. Context-sensitive call graph construction ($k = 1$ PTA) provides marginal precision improvement at 5.02 $\times$ overhead (46,870 ms vs. 9,344 ms for CHA, across 1,001 projects with 1,701,662 virtual call sites); $k = 2$ is faster

**Figure 5: Large-scale corpus landscape and category-sink interactions.**

(34,473 ms) than $k = 1$, likely because the deeper context reduces the number of spurious call targets and thus the work per call site, though we did not verify this hypothesis. The full configuration (IFDS+CB+LC+PTA) achieves 91.6% precision and 96.3% recall (F1 93.9%).

5.6 RQ4: Corpus-level Privacy Patterns

Figure 5 shows the long-tail distribution, category-sink interactions, and scale correlations. The corpus is strongly skewed: 236/1,015 projects contain detected privacy APIs, and the median project contains zero detected APIs. The top 10 projects account for 546/1,757 API usages (31.08%), the top 50 account for 64.14%, and the ArkPrismBench sample covers 925/1,757 (52.65%).

The corpus is strongly skewed: 236/1,015 projects contain detected privacy APIs. Device identity (screen/display and hardware) dominates at 45.25% of API usages, followed by network connectivity (8.42%), clipboard (6.55%), and location (5.81%). Project size partly explains privacy evidence (Pearson $r = 0.51$ for methods vs. API usages) but is not a reliable proxy for privacy behavior. See Table 13 in Appendix B.6 for the full per-category breakdown.

Additional benchmark methodology details, including annotation protocol, zero-output statistical bounds, and inter-annotator agreement, are in Appendix B.7. Performance and scalability results are in Appendix B.8.

6 Discussion

6.1 Implications of the Results

The results show that ARKPRISM scales to a large OpenHarmony corpus and recovers useful evidence for privacy-sensitive API recognition and information-flow subgraph mapping. The strongest engineering evidence is end-to-end coverage: all 1,015 projects produced reports, and each detected API usage received a non-empty call-chain entry. Package-aware matching substantially reduces the noise of raw method search, while the ArkPrismBench stratified benchmark confirms detection accuracy across all output levels, including zero-output projects.

The results should be interpreted at the evidence level. A taint flow to a log or UI display is not automatically a privacy violation; a local fallback chain is not a complete lifecycle path; the benchmark covers only 16.3% of the corpus and cannot guarantee zero false negatives in unsampled projects (Section B.7). ARKPRISM is designed as an audit-support system that preserves API, call-chain, sink, taint, and metadata evidence for downstream auditors. The design also exposes natural ablation points—package-family normalization, namespace-member alias tracking, indirect-call recognition, typed callback edges, lifecycle modeling, and context-sensitivity level k can each be disabled independently, supporting both high-recall marketplace auditing and stricter continuous-integration settings.

6.2 Threats to Validity

Benchmark and corpus validity. The ArkPrismBench stratified benchmark addresses purposive-sampling bias by including zero-output and low-output projects, but covers only 16.3% of the corpus; the micro-benchmark provides controlled pattern-level validation but uses hand-crafted cases. The corpus contains real projects but also many demos and samples, which can inflate counts for sensors, camera, network, and clipboard APIs. Additional evaluations on marketplace and industrial corpora would strengthen external validity.

Analysis validity. Call-chain validity is limited by the separation of framework-entry, initialization, unknown-entry, and local fallback chains—chain-entry coverage should not be read as complete framework-entry recovery. Taint validity depends on SDK signature resolution, pointer analysis, and callback modeling; callback-specific flows still require manual inspection. Lifecycle modeling does not increase taint flow counts even at Level 2 because IFDS's may-analysis explores all inter-procedural paths regardless of execution order; only 3/302 projects with taint flows show any count change (each losing one flow from more precise call resolution). The lifecycle model's contribution is instead in collaboration behavior (+44.2% in quantity, 260→375 projects; precision not systematically audited) and call graph enrichment (+32.6% average CG node increase). Context-sensitive CG ($k = 1$) provides marginal precision improvement at 5.02× overhead (96.3%/91.6% vs. 97.8%/91.1%), with recall affected by 5 estimated false negatives from `@kit.*` namespace imports not currently matched by IFDS source resolution. The lifecycle model's cross-layer transitions depend on ability/component class discovery, which may miss custom subclasses.

Rule and temporal validity. The sensitive API rule file reflects our current understanding; some entries may be over- or under-inclusive, affecting both detection and benchmark construction. The results are tied to a fixed corpus, rule set, and SDK version—future comparisons should report the analyzed configuration explicitly.

6.3 Limitations

ARKPRISM is static and source-based. It cannot observe runtime-only configuration, dynamically loaded modules, server-provided policy, platform permission prompts, or user consent. It also cannot determine whether a privacy-policy disclosure is adequate. The graph it produces is an evidence artifact for audit, not a final compliance verdict. Dynamic confirmation, policy-text analysis, and marketplace-scale deployment are natural extensions.

7 Related Work

7.1 Source Identity and Privacy API Recognition

Accurate source/sink specifications are critical for mobile privacy analysis. Android studies have mined permission mappings and source/sink lists via Stowaway, PScout, SuSi, Explorer, and API-documentation analyses [7, 8, 20, 40, 42], and audited privacy behavior by connecting code, policies, and permissions [3, 50, 56, 57]. These works show that source/sink definitions encode platform semantics and strongly affect reported results.

ARKPRISM applies this principle to OpenHarmony, where package migration from `@ohos.*` to `@kit.*` compounds the challenge. Its matching function separates conceptual API identity from exact package spelling. HapFlow's `Json2ArkMethodSignature` matches by namespace and method name against SDK declarations, but cannot recognize APIs invoked through manager/helper receivers or privacy-constant fields [16]—an architectural blind spot of method-signature matching. ArkAnalyzer provides ArkTS code representation and call-graph construction [15] but does not address API identity resolution. ARKPRISM's package- and receiver-aware *Id(o)* fills this gap.

7.2 Asynchronous Semantics and Lifecycle Modeling

Mobile analysis must model lifecycle, callbacks, and implicit edges precisely. Android ICC modeling includes ComDroid, CHEx, Epicc, IC3, IccTA, DidFail, and RAICC [18, 29, 31, 35–37, 45]; EdgeMiner recovers implicit framework callbacks [13]. Hybrid-app analyses bridge Java, JavaScript, and WebView semantics [11, 30, 52, 53]. These studies show that missing implicit edges dominate mobile analysis errors.

OpenHarmony has analogous but different callback problems: ArkUI declarative builders, class-field callbacks, `FunctionType/ClosureType` values, and `Promise` handlers create non-method-call edges. HapFlow handles closure semantics [16], but `Promise.then()` chains and builder-option callbacks remain outside its source-binding model. ARKPRISM's asynchronous source binding *SrcBind(call, callback, param)* (T1–T5) follows the same principle as Android ICC modeling, instantiated for ArkTS IR and OpenHarmony event semantics. The lifecycle state machine (Section 3.6)

captures ability/component phase transitions that the flat Dummy-Main cannot express.

7.3 Evidence Representation and Large-scale Auditing

Android taint analysis has a rich history. TaintDroid [19] motivated smartphone privacy monitoring; FlowDroid [6] introduced context-/flow-/field-/object-sensitive lifecycle-aware analysis; Amandroid, DroidSafe, IccTA, LeakMiner, and AndroidLeaks explored inter-procedural data-flow, vetting, and market-side detection [24, 25, 31, 48, 55]; HornDroid and COVERT added formal and compositional perspectives [9, 12]; DroidBench and comparative studies exposed precision/recall tradeoffs [5, 58]. ARKPRISM uses HapFlow as the OpenHarmony taint substrate and adds recognition and subgraph evidence around it.

Malware and behavior research uses permissions, API calls, graphs, and models to classify malicious apps [1, 4, 21, 23, 38, 41, 46, 49, 51, 54, 59–61]; AndroZoo enables ecosystem-scale studies [2]. ARKPRISM follows this empirical principle: the information-flow subgraph is more structured than a flat feature vector because auditors need source snippets, call contexts, sink classes, and metadata.

ARKPRISM builds on IFDS/IDE foundations [43, 44] and reusable infrastructures (Soot, WALA, Heros [10, 22, 47]). ArkAnalyzer plays this role for ArkTS [15]. Other OpenHarmony efforts explore testing [17, 34], repair [33], and pointer analysis [14], showing that OpenHarmony requires ecosystem-specific analysis [27, 32, 39]. ARKPRISM follows the same layered principle: reuse the framework, then add domain-specific recognition and provenance-annotated subgraph mapping.

8 Conclusion

We presented ARKPRISM, a static analysis framework that addresses the source identity resolution bottleneck in ArkTS privacy analysis. While HapFlow solves taint propagation, ARKPRISM solves the preceding problem: reliably identifying which API calls are privacy-sensitive, resolving their identities across package migration and receiver factories, and binding them to asynchronous consumers. The core contributions are package- and receiver-aware API identity resolution $Id(v)$, asynchronous source binding $SrcBind(call, callback, param)$, and provenance-annotated evidence subgraphs. On the ArkPrismBench stratified benchmark (164 projects), ARKPRISM achieves 100%/100% precision/recall; HapFlow's method-signature matching detects at most 68.7% of gold usages. Callback analysis is the dominant component, improving taint flow recall from 51.5% to 97.8% ($p < 0.001$). The full pipeline analyzes 1,015 projects in ≈ 4.2 h wall time with 99.8% completion, providing scalable privacy audit evidence with full provenance transparency.

Data Availability

Our artifact includes: (1) the ArkPrism tool source code and build scripts, (2) the ARGUS-1015 dataset of 1,015 HarmonyOS applications with per-project analysis reports, (3) the ArkPrismBench stratified benchmark (164 projects) with ground-truth annotations,

(4) the ArkPrismBench-Micro (77 test cases across 15 pattern categories), (5) source/sink configuration files, and (6) scripts for reproducing all evaluation tables. We will provide an anonymized artifact package upon acceptance.

References

- [1] Yousra Aafer, Wenliang Du, and Heng Yin. 2013. DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android. In *Proceedings of the International Conference on Security and Privacy in Communication Systems*. 86–103.
- [2] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *Proceedings of the 13th International Conference on Mining Software Repositories (ICMSR)*. ACM, New York, NY, USA, 468–471.
- [3] Benjamin Andow, Samin Yaseer Mahmud, Justin Whitaker, William Enck, Bradley Reaves, Kapil Singh, and Serge Egelman. 2020. Actions Speak Louder than Words: Entity-Sensitive Privacy Policy and Data Flow Analysis with PoliCheck. In *29th USENIX Security Symposium (USENIX Security)*. USENIX Association, Online, 985–1002.
- [4] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, Konrad Rieck, and CERT Siemens. 2014. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In *Proceedings of the Network and Distributed System Security Symposium*.
- [5] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Ocaeu, and Patrick McDaniel. 2014. A Benchmark Suite for Static Taint Analysis of Android Applications. In *Proceedings of the ACM SIGPLAN International Workshop on State of the Art in Java Program Analysis*. 27–30.
- [6] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Ocaeu, and Patrick McDaniel. 2014. FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery, New York, NY, USA, 259–269.
- [7] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. PScout: Analyzing the Android Permission Specification. In *Proceedings of the ACM Conference on Computer and Communications Security*. 217–228.
- [8] Michael Backes, Sven Bugiel, Erik Derr, Patrick McDaniel, Damien Ocaeu, and Sebastian Weisgerber. 2016. On Demystifying the Android Application Framework: Re-Visiting Android Permission Specification Analysis. In *Proceedings of the USENIX Security Symposium*. 1101–1118.
- [9] Hamid Bagheri, Alireza Sadeghi, Joshua Garcia, and Sam Malek. 2015. COVERT: Compositional Analysis of Android Inter-App Permission Leakage. In *Proceedings of the IEEE/ACM International Conference on Software Engineering*. 866–877.
- [10] Eric Bodden. 2012. Inter-procedural Data-flow Analysis with IFDS/IDE and Soot. In *Proceedings of the ACM SIGPLAN International Workshop on State of the Art in Java Program Analysis*. 3–8.
- [11] Achim D. Brucker and Michael Herzberg. 2018. BabelView: Evaluating the Impact of Code Injection Attacks in Mobile Webviews. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. 25–46.
- [12] Stefano Calzavara, Ilya Grishchenko, and Matteo Maffei. 2016. HornDroid: Practical and Sound Static Analysis of Android Applications by SMT Solving. In *Proceedings of the IEEE European Symposium on Security and Privacy*. 47–62.
- [13] Yinzi Cao, Yanick Fratantonio, Antonio Bianchi, Antonio Chung, Yanick Huang, David Lie, Giovanni Vigna, and Christopher Kruegel. 2015. EdgeMiner: Automatically Detecting Implicit Control Flow Transitions through the Android Framework. In *Proceedings of the Network and Distributed System Security Symposium*.
- [14] Haonan Chen et al. 2026. Context-Sensitive Pointer Analysis for ArkTS. arXiv preprint. <https://arxiv.org/abs/2602.00457>
- [15] Haonan Chen, Daihang Chen, Yizhuo Yang, Lingyun Xu, Liang Gao, Mingyi Zhou, Chunming Hu, and Li Li. 2025. ArkAnalyzer: The Static Analysis Framework for OpenHarmony. In *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE SEIP)*. ACM.
- [16] Haonan Chen, Mingyi Zhou, Yanjie Zhao, and Li Li. 2026. HapFlow: The Taint Analysis Framework for OpenHarmony Apps. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3793863
- [17] Yige Chen, Yujia Fan, Siyi Wang, Yi Tao, and Yepang Liu. 2025. HmTest: Automated Testing of HarmonyOS Apps via Model-Driven Navigation and Reinforcement Learning. *Journal of Computer Science and Technology* 40, 4 (2025), 1006–1021. doi:10.1007/s11390-025-5142-4
- [18] Erika Chin, Adrienne Porter Felt, Kate Greenwood, and David Wagner. 2011. Analyzing Inter-Application Communication in Android. In *Proceedings of the 9th International Conference on Mobile Systems, Applications, and Services*. 239–252.

- [19] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyoon Jung, Patrick McDaniel, and Anmol N. Sheth. 2014. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. *ACM Trans. Comput. Syst.* 32, 2 (2014), 5:1–5:29.
- [20] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. 2011. Android Permissions Demystified. In *Proceedings of the ACM Conference on Computer and Communications Security*. 627–638.
- [21] Yu Feng, Saswat Anand, Isil Dillig, and Alex Aiken. 2014. Apposcopy: Semantics-Based Detection of Android Malware through Static Analysis. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 576–587.
- [22] Stephen J. Fink, Julian Dolby, and Eran Yahav. 2008. Effective Typestate Verification in the Presence of Aliasing. *ACM Transactions on Software Engineering and Methodology* 17, 2, Article 9 (2008).
- [23] Yanick Fratantonio, Antonio Bianchi, William Robertson, Dhilung Kirat, Christopher Kruegel, and Giovanni Vigna. 2016. TriggerScope: Towards Detecting Logic Bombs in Android Applications. In *Proceedings of the IEEE Symposium on Security and Privacy*. 377–396.
- [24] Clint Gibler, Jonathan Crussell, Jeremy Erickson, and Hao Chen. 2012. AndroidLeaks: Automatically Detecting Potential Privacy Leaks in Android Applications on a Large Scale. In *Proceedings of the 5th International Conference on Trust and Trustworthy Computing*. 291–307.
- [25] Michael I. Gordon, Deokhwan Kim, Jeff H. Perkins, Limei Gilham, Nguyen Nguyen, and Martin Rinard. 2015. Information Flow Analysis of Android Applications in DroidSafe. In *Proceedings of the Network and Distributed System Security Symposium*.
- [26] James A. Hanley and Abby Lippman-Hand. 1983. If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators. *Journal of the American Medical Association* 249, 13 (1983), 1743–1745.
- [27] Huawei. 2021. HarmonyOS 2 Security Technical White Paper. <https://consumer.huawei.com/>
- [28] Huawei Developer. 2026. ArkTS: Dedicated Programming Language for the HarmonyOS Ecosystem. <https://developer.huawei.com/consumer/en/arkts/>
- [29] William Klieber, Lori Flynn, Amar Bhosale, Limin Jia, and Lujo Bauer. 2014. Android Taint Flow Analysis for App Sets. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 1–6.
- [30] Sungho Lee, Julian Dolby, and Sukyoung Ryu. 2016. HybriDroid: static analysis framework for Android hybrid applications. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Association for Computing Machinery, New York, NY, USA, 250–261. doi:10.1145/2970276.2970368
- [31] Li Li, Alexandre Bartel, Tegawendé F. Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Oeteau, and Patrick McDaniel. 2015. IccTA: Detecting Inter-Component Privacy Leaks in Android Apps. In *Proceedings of the 37th International Conference on Software Engineering*. 280–291.
- [32] Li Li, Xiang Gao, Hailong Sun, Chunming Hu, Xiaoyu Sun, Haoyu Wang, Haipeng Cai, Ting Su, Xiapu Luo, Tegawendé F. Bissyandé, Jacques Klein, John Grundy, Tao Xie, Haibo Chen, and Huaimin Wang. 2025. Software Engineering for OpenHarmony: A Research Roadmap. *Comput. Surveys* 58, 2, Article 34 (2025), 34 pages. doi:10.1145/3720538
- [33] Zhihao Lin, Mingyi Zhou, Wei Ma, Chi Chen, Yun Yang, Jun Wang, Chunming Hu, and Li Li. 2025. HapRepair: Learn to Repair OpenHarmony Apps. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Industry Track*. 319–330. doi:10.1145/3696630.3728556
- [34] Farong Liu, Mingyi Zhou, Yakun Zhang, Ting Su, Bo Sun, Jacques Klein, Xiang Gao, and Li Li. 2025. HapTest: The Dynamic Analysis Framework for OpenHarmony. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Companion Proceedings*. 1–10. doi:10.1145/3696630.3728565
- [35] Long Lu, Zhichun Li, Zhenyu Wu, Wenke Lee, and Guofei Jiang. 2012. CHEX: Statically Vetting Android Apps for Component Hijacking Vulnerabilities. In *Proceedings of the ACM Conference on Computer and Communications Security*. 229–240.
- [36] Damien Oeteau, Somesh Jha, and Patrick McDaniel. 2013. Effective Inter-Component Communication Mapping in Android with Epic: An Essential Step Towards Holistic Security Analysis. In *Proceedings of the 22nd USENIX Security Symposium*. 543–558.
- [37] Damien Oeteau, Daniel Luchaup, Matthew Dering, Somesh Jha, and Patrick McDaniel. 2015. Composite Constant Propagation: Application to Android Inter-Component Communication Analysis. In *Proceedings of the 37th International Conference on Software Engineering*. 77–88.
- [38] Lucky Onwuzurike, Enrico Mariconti, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2019. MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models (Extended Version). *ACM Transactions on Privacy and Security (TOPS)* 22, 2, Article 10 (2019), 10:1–10:29 pages.
- [39] OpenAtom Foundation. 2026. OpenHarmony: A Comprehensive Open Source Project for the All-scenario, Fully-connected, Intelligent Era. <https://www.openharmony.cn/>
- [40] Siegfried Rasthofer, Steven Arzt, and Eric Bodden. 2014. A Machine-learning Approach for Classifying and Categorizing Android Sources and Sinks. In *Proceedings of the Network and Distributed System Security Symposium*.
- [41] Siegfried Rasthofer, Steven Arzt, Enrico Lovat, and Eric Bodden. 2016. HAR-VESTER: Automatic Generation of Entry Points for Dynamic Analysis of Android Applications. In *Proceedings of the Network and Distributed System Security Symposium*.
- [42] Talia Ravitch, Eric Creswick, Aaron Tomb, Adam Foltzer, Trevor Elliott, and Ledah Casburn. 2014. Multi-App Security Analysis with Fuse: Statically Detecting Android App Collusion. In *Proceedings of the ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 4–9.
- [43] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 49–61.
- [44] Mooly Sagiv, Thomas Reps, and Susan Horwitz. 1996. Precise Interprocedural Dataflow Analysis with Applications to Constant Propagation. *Theoretical Computer Science* 167, 1–2 (1996), 131–170.
- [45] Jordan Samhi, Alexandre Bartel, Tegawendé F. Bissyandé, and Jacques Klein. 2021. RAICC: Revealing Atypical Inter-Component Communication in Android Apps. *Empirical Software Engineering* 26, 6 (2021).
- [46] Kimberly Tam, Ali Feizollah, Nor Badrul Anuar, Rosli Salleh, and Lorenzo Cavallo. 2015. CopperDroid: Automatic Reconstruction of Android Malware Behaviors. In *Proceedings of the Network and Distributed System Security Symposium*.
- [47] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundareshan. 2010. Soot: A Java Bytecode Optimization Framework. In *CASCON First Decade High Impact Papers*. IBM Corp., 214–224.
- [48] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2018. Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. *ACM Trans. Priv. Secur.* 21, 3 (2018), 14:1–14:32.
- [49] Michelle Y. Wong and David Lie. 2016. IntelliDroid: A Targeted Input Generator for the Dynamic Analysis of Android Malware. In *Proceedings of the Network and Distributed System Security Symposium*.
- [50] Meng Xia, Lu Gong, Yuan Lyu, Zhengwei Qi, and Xue Liu. 2015. Effective Real-time Android Application Auditing. In *Proceedings of the IEEE Symposium on Security and Privacy*. 899–914.
- [51] Lok Kwong Yan and Heng Yin. 2012. DroidScope: seamlessly reconstructing the OS and Dalvik semantic views for dynamic Android malware analysis. In *Proceedings of the 21st USENIX Conference on Security Symposium (USENIX Security)* (Bellevue, WA). USENIX Association, USA, 29.
- [52] Guangliang Yang, Abner Mendoza, and Guofei Gu. 2018. Automated Generation of Event-Oriented Exploits in Android Hybrid Apps. In *Proceedings of the Network and Distributed System Security Symposium*.
- [53] Guangliang Yang, Abner Mendoza, Jialong Zhang, and Guofei Gu. 2017. BridgeScope: Precisely and Scalably Vetting JavaScript Bridge Security Issues in Android Hybrid Apps. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. 166–185.
- [54] Wei Yang, Xusheng Xiao, Benjamin Andow, Sihan Li, Tao Xie, and William Enck. 2015. AppContext: Differentiating Malicious and Benign Mobile App Behaviors Using Context. In *Proceedings of the International Conference on Software Engineering*. 303–313.
- [55] Zheming Yang and Min Yang. 2012. LeakMiner: Detect Information Leakage on Android with Static Taint Analysis. In *Proceedings of the 3rd World Congress on Software Engineering*. 101–104.
- [56] Zheming Yang, Min Yang, Yuan Zhang, Guofei Gu, Peng Ning, and X. Sean Wang. 2013. AppIntent: Analyzing Sensitive Data Transmission in Android for Privacy Leakage Detection. In *Proceedings of the ACM Conference on Computer and Communications Security*. 1043–1054.
- [57] Le Yu, Xiapu Luo, Jiachi Chen, Hao Zhou, Tao Zhang, Henry Chang, and Hareton K. N. Leung. 2021. PPChecker: Towards Accessing the Trustworthiness of Android Apps' Privacy Policies. *IEEE Transactions on Software Engineering (TSE)* 47, 2 (2021), 221–242.
- [58] Jun Zhang, Yang Feng, Li Li, Tegawendé F. Bissyandé, and Jacques Klein. 2021. Analyzing Android Taint Analysis Tools: FlowDroid, Amandroid, and DroidSafe. *IEEE Transactions on Software Engineering* 47, 11 (2021), 2395–2414.
- [59] Mu Zhang, Yue Duan, Heng Yin, and Zhiruo Zhao. 2014. Semantics-Aware Android Malware Classification Using Weighted Contextual API Dependency Graphs. In *Proceedings of the ACM Conference on Computer and Communications Security*. 1105–1116.
- [60] Wu Zhou, Yajin Zhou, Xuxian Jiang, and Peng Ning. 2012. Detecting Repackaged Smartphone Applications in Third-party Android Marketplaces. In *Proceedings of the ACM Conference on Data and Application Security and Privacy*. 317–326.
- [61] Yajin Zhou, Zhi Wang, Wu Zhou, and Xuxian Jiang. 2012. DroidRanger: Systematic Characterization and Detection of Malicious Applications in Official and Alternative Android Markets. In *Proceedings of the Network and Distributed System Security Symposium*.

A Supplementary Details

A.1 Design Map

Table 8 summarizes the notation used throughout the paper. Table 9 maps each analysis layer to the observed gap, formal object, optimization, and stored subgraph evidence.

Table 8: Notation.

Symbol	Meaning	Produced by
P	An analyzed OpenHarmony project	Scene builder
R	Privacy-sensitive API rules	Rule loader
$Id(v)$	Semantic API identity $\langle Pkg, Ns, Recv, Mem, Acc \rangle$	API resolver
A	Import and IR alias relation	API resolver
$SrcBind$	Asynchronous source binding ($call, cb, param$)	Callback analyzer
U	Recognized privacy-sensitive API usages	Match engine
C_u	Recovered call-chain set for usage u	Chain tracer
K_u	Downstream sink set attached to u	Sink analyzer
T_u	Taint paths related to u	HapFlow runner
$\omega(e)$	Provenance label for edge e	Graph assembler
G_u	Information-flow subgraph for u	Graph assembler

A.2 Failure Modes Observed in a Large-scale Corpus

Before designing ArkPRISM, we repeatedly ran the ArkAnalyzer/HapFlow-based pipeline on the large-scale dataset and inspected failures. The most common failure modes were:

- **Layout misses.** Some projects contained valid ArkTS source files but were not discovered by standard project construction.
- **Package migration misses.** Rules written for `@kit.*` packages missed apps importing equivalent historical `@ohos.*` packages, and vice versa.
- **IR-lowering misses.** Multi-segment API expressions were lowered into temporaries, causing the final method call to lose its namespace context.
- **Indirect receiver misses.** Manager APIs were called through objects whose types were absent, imprecise, or not directly tied to the original namespace.
- **Callback-edge misses.** Anonymous functions and field-stored callbacks were not always connected to their registration sites in the recovered call chain.
- **Callback taint loss.** Runs without callback/Promise taint propagation undercounted or eliminated taint flows even when source and sink rules loaded correctly.

These failure modes motivate the design in Section 3 and provide natural ablation points: each optimization can be disabled to measure its effect on API recall, chain coverage, sink attachment, or taint-flow count.

B Supplementary Evaluation Details

B.1 Context-Sensitive Call Graph Details

Context-insensitive call graphs (CHA, RTA) merge all calling contexts of a method into a single node, causing spurious data flow at merge points. ArkPRISM supports k -limited context-sensitive call graph construction using ArkAnalyzer’s pointer analysis infrastructure. The context sensitivity level k controls the depth of calling context tracked for each method invocation. At $k = 0$, the analysis is context-insensitive (equivalent to CHA). At $k = 1$, each call site receives its own context, which is sufficient to separate most callback-related data flows. At $k = 2$, calling contexts are tracked two levels deep, which helps with nested callback patterns.

B.2 Representative Collaboration Behavior Projects

Table 11 shows representative projects with the largest collaboration behavior increases under lifecycle-enhanced CG.

B.3 Detector Contribution and Leave-One-Out Ablation

B.4 Micro-benchmark Detailed Results

Three failure modes are isolated in the ArkPrismBench-Micro: (1) M08 (Promise `.then()`) misses 2 positive cases where the callback is defined in a separate method from the `.then()` call—our current Promise chain analysis only handles inline and variable-chained callbacks; (2) M06 (same-name negative) produces 2 false positives where user-defined methods sharing names with privacy APIs (`getCurrentLocation()`) are incorrectly matched—this motivates receiver-type-aware filtering; (3) M07 (string/comment) correctly rejects all cases, confirming that the recognizer does not match non-executable references.

Aggregating the asynchronous-pattern categories (M08–M13), callback target resolution achieves 18/20 correct targets (90.0% resolution accuracy), and tainted-parameter binding achieves 17/20 (85.0%). Promise edge precision is 100.0% (2/2), while Promise edge recall is 50.0% (2/4), reflecting the cross-method gap.

Table 4 (Section 5.3) decomposes detections by detector category, reports a cumulative coverage view on the full corpus, and provides a leave-one-out ablation on the ArkPrismBench manual benchmark. The leave-one-out ablation confirms that each family is irreplaceable: removing any single family drops method recall below 86% and usage recall below 84%.

B.5 Call-Chain Class Distribution

Table 12 reports the distribution of call-chain classes across the 1,015-project corpus.

Most chains are short: 982 have length 1, 642 have length 2–3, 103 have length 4–5, and 30 have length 6–10. Entry-type counts further support this: 979 chains are component-lifecycle entries, 324 initialization entries, 125 app-lifecycle entries, 1 user-interaction entry, and 328 unknown entries.

Table 9: Design map.

Layer	Observed gap	Effect without ARKPRISM	Formal object	ARKPRISM optimization	Subgraph evidence
Rules Imports	@ohos/@kit migration Default/namespace aliases	Package-sensitive misses Namespace loss	$\eta(pkg), Pkg$ A_{import}, Ns	Package-family normalization Import alias map and lower-case variants	Stable API key Matched namespace
IR	Lowered multi-segment calls	Lost calls such as <code>request.agent.create</code>	A_{lr}	Namespace-member alias tracking	Recovered signature
Receiver	Manager/factory objects	Indirect calls invisible	$Recv$	Bounded backward data-flow tracing	Receiver provenance
Property	Device-identity fields	Non-method sources invisible	$Acc=property$	Method/property unified representation	Field access label
Callbacks	ArkUI closures and fields	Truncated chains	$SrcBind(call, cb, param)$	Callback resolution + Promise chains	Chain class
Provenance Taint	No derivation tracking Callback/Promise data	Unevaluable evidence Empty or partial IFDS flows	$\omega(e)$ T_u	Per-edge provenance label Default callback/Promise propagation	Confidence Taint paths

Table 10: Context-sensitive call graph construction (k -CFA, 1,001/1,015 projects).

k	CG nodes	Virtual sites	PTA time	Overhead
0 (CHA)	419,462	1,701,662	—	baseline
0 (RTA)	418,776	1,701,662	7,383 ms	0.79×
1	—	1,701,662	46,870 ms	5.02×
2	—	1,701,662	34,473 ms	3.69×

Table 11: Representative projects with collaboration behavior increases.

Project	Flat CG	LC CG	ΔCG	Flat Collab	LC Collab
legado	43,450	81,486	+87.5%	3	7
BikeTravel	877	1,128	+28.6%	1	3
Aigis	1,575	1,863	+18.3%	2	3
Application	2,105	2,922	+38.8%	1	2
Snake_NEXT	2,971	3,290	+10.7%	3	5

B.6 Top Privacy Categories and Package Families

Table 13 reports the top privacy categories and their dominant package families across the 1,015-project corpus.

The top package families are @kit.ArkUI (438 usages), @kit.BasicServicesKit (376), @ohos.deviceInfo (175), @kit.NetworkKit (105), @kit.ConnectivityKit (93), and @kit.LocationKit (87). The most frequent methods are `display.Display.getDefaultDisplaySync` (422), `deviceType` (283), `getSystemPasteboard` (74), and `getCurrentLocation` (54).

B.7 Manual Audit Details

The ArkPrismBench benchmark uses stratified random sampling rather than purposive selection, ensuring that zero-output and low-output projects are represented. However, the sampling strata are defined by ARKPRISM’s raw API count, and ARKPRISM’s reports serve as navigation aids during annotation; the benchmark is therefore *not fully independent* of the tool. The annotation itself is source-first: annotators label based on code inspection, not tool output, but the tool’s reports influence which code regions receive the most attention.

Recall scope. Recall is measured against the manually accepted project-level namespace+method gold set for all 164 sampled projects. For the zero-output stratum (40 projects), annotators confirmed that no privacy-sensitive API usage exists in the source code, verifying that ARKPRISM did not miss any APIs in these projects. For the positive strata, annotators identified all privacy API usages in the source, including those ARKPRISM may have missed.

Zero-output statistical bound. The 40 zero-output projects represent a sample from 802 zero-output projects in the corpus. While no false negatives were found in the sample, the rule-of-three method [26] gives a 95% confidence upper bound of $3/40 = 7.5\%$ on the false-negative rate among unsampled zero-output projects. We therefore cannot claim that all 802 zero-output projects are FN-free; we can only claim zero FN within the sampled 40.

Negative benchmark coverage. The zero-output stratum covers 40 projects where ARKPRISM detected nothing. Manual inspection confirmed that none contained a privacy API usage matching any of the 21 categories. This check covers direct-call, property-read, and indirect-invoke patterns (the latter through manual code inspection rather than grep, since indirect-invoke patterns require semantic resolution).

Annotation quality. Two annotators independently labeled each project. Inter-annotator agreement on the positive strata was $\kappa = 0.91$ (Cohen’s kappa), with disagreements resolved by joint re-inspection. Common disagreement sources were: helper receiver calls where the API is accessed through a factory-returned object, and property constants whose names resemble ordinary UI fields.

B.8 Performance and Scalability

We report wall-clock runtime, stability, and resource characteristics on the full 1,015-project corpus. All measurements are from the isolated batch run (Section 5.2) using two worker processes on a machine with 16 GB RAM and an 8-core CPU. Each project is analyzed in a separate child process with a 30-minute timeout and a 15-minute IFDS budget per project. We report CPU-seconds (cumulative single-worker time) for per-project statistics, and estimated wall-clock time for the full batch.

The runtime distribution is right-skewed: 75% of projects complete within 32.7 s, and 95% within 39 s. The longest project (aloeplayer_ohos, 270.8 s) is an outlier with 10 privacy APIs and

Table 12: Call-chain classes.

Chain class / metric	Count	Share	Semantics	Confidence
Total call-chain entries	1,757	100.00%	One entry for every recognized API usage	complete entry coverage
Framework-entry chains	1,105	62.89%	Reaches a framework, lifecycle, or UI-style entry	high
Initialization chains	324	18.44%	Starts from %dflt, %statInit, %instInit, or render initialization	medium
Local fallback chains	280	15.94%	Declaring-method evidence without trusted predecessor	local evidence
Unknown-entry chains	48	2.73%	Non-empty chain with unresolved entry semantics	medium-low
Chains with at least one sink	839	47.75%	API usage has downstream sink evidence	sink-linked
Chain length median / P95 / max	1 / 4 / 9	–	Short paths dominate; long chains remain auditable	descriptive

Table 13: Top privacy categories and package families.

Privacy category	Count	Share	Dominant package family	Typical evidence
Device identity: screen/display	438	24.93%	@kit.ArkUI	display size, density, orientation, default display.
Device identity: hardware	357	20.32%	@kit.BasicServicesKit, @ohos.deviceInfo	model, type, brand, hardware identity fields.
Network connectivity	148	8.42%	@kit.NetworkKit	default network, capabilities, connection properties.
Clipboard	115	6.55%	@kit.BasicServicesKit, @ohos.pasteboard	system pasteboard access and paste data.
Location	102	5.81%	@kit.LocationKit	current/last location and reverse geocoding.
Bluetooth	85	4.84%	Connectivity kits	Bluetooth state, scan, profile, and advertising APIs.
Sensor status	76	4.33%	@kit.SensorServiceKit	sensor subscription and single-sensor queries.
Wi-Fi	69	3.93%	@kit.ConnectivityKit, @ohos.wifiManager	Wi-Fi state, scans, and linked information.
Device software identity	64	3.64%	Basic services	OS name, OS version, API version fields.
Media	55	3.13%	Media libraries	asset creation, media access, metadata.
Camera	49	2.79%	@kit.CameraKit	camera manager and camera capability queries.
Audio status	41	2.33%	@kit.AudioKit	audio manager and volume group APIs.

Table 14: Runtime and stability on the 1,015-project corpus.

Metric	Value	Detail
Runtime (CPU-seconds)		
Total	30,142 s	≈8.4 h single-worker
Median	30.7 s	per project
P90	36.2 s	
P95	39.0 s	
Max	270.8 s	aloeplayer_ohos
Stability		
Timeouts	0	30-min limit
OOM / budget exceeded	0	15-min IFDS limit
Complete analyses	1,013/1,015	99.8%
Partial (solver failure)	2	Aigis, libretro-emulator
PTA warnings	11	non-fatal pointer-analysis issues
Call-graph construction		
CHA (k=0) total	9,344 ms	1,001 projects
PTA (k=1) total	46,870 ms	5.02× CHA
PTA (k=2) total	34,473 ms	3.69× CHA

a large method count that triggers extensive IFDS propagation. The two solver failures are caused by unresolvable ArkTS IR structures in those projects; they do not affect other projects because of process isolation.

The dominant cost is IFDS taint analysis, which accounts for the majority of per-project time. Scene construction (ArkAnalyzer IR

loading) is the second-largest cost, typically consuming 40–60% of the total. Call-graph construction (CHA) is negligible at a median of 3 ms per project. Callback analysis adds minimal overhead because it operates on the already-constructed IR. PTA at $k = 1$ adds 5.02× overhead relative to CHA alone, but CHA itself is a small fraction of total runtime; the effective overhead of PTA over the full pipeline is therefore modest. The fact that $k = 2$ is faster than $k = 1$ (34,473 ms vs. 46,870 ms) is counter-intuitive; a likely explanation is that the deeper context reduces the number of spurious call targets per site, lowering the total work despite the larger context depth.

The estimated two-worker wall time for 1,015 projects is 15,071 s (≈4.2 h), suggesting near-linear scaling with worker count. The process-per-project isolation strategy prevents memory accumulation across projects, but imposes a per-project startup cost. For a hypothetical 10,000-project marketplace corpus, the projected wall time with 4 workers is ≈8.4 h, well within overnight-batch feasibility.